

Study of LoRaWAN device and gateway setups with ChirpStack implementation

Fredrik Lund

Bachelor of Science Thesis in Electrical Engineering

Study of LoRaWAN device and gateway setups

Fredrik Lund

LiTH-ISY-EX-ET--22/0512--SE

Supervisors:

ISY, Linköping University

Examiner:

J Jacob Wikner

ISY, Linköping University

*Division of Integrated Circuits and Systems
Department of Electrical Engineering
Linköping University
SE-581 83 Linköping, Sweden*

Copyright 2022 Fredrik Lund

Abstract

This document presents basic theory about how to setup a network solution for transferring sensor data and also gives instructions on how to set up some of its building blocks. This document mainly handles LoRa, LoRaWAN, Chirpstack, Arduino and some other related frameworks.

Sammanfattning

Denna anvisning går igenom grundläggande teorier om hur man sätter upp en nätverkslösning för att kommunicera sensor data och ger även instruktioner för hur man sätter upp några av dessa enheter. Det här dokumentet handlar om LoRa, LoRAWAN, Chirpstack, Arduino och några andra relaterade ramverk.

Acknowledgments

I want to thank my family and my fiancée, the best Marielle, who has accepted different hardware all over the apartment. I also want to thank my examiner, Jacob for his support and understanding when doing this work.

Disclaimer

This thesis work is part of the BRIGHT project. BRIGHT is funded by the Swedish Research Council for Sustainable Development Formas, under the call “Climate adaptation of the built environment 2021”.

Contents

1 Introduction.....	3
1.1 Motivation.....	5
1.2 Purpose.....	5
1.3 Problem statements.....	5
1.4 (Research) Limitations.....	6
2 Background.....	7
2.1 Introduction.....	7
2.2 Conclusions.....	8
3 Theory.....	9
3.1 Introduction.....	9
3.2 Long-range communication (LoRa).....	12
3.2.1 LoRa frequencies.....	12
3.2.2 LoRa modulation.....	13
3.2.3 Spreading factor (SF).....	13
3.2.4 Free Space Path loss.....	14
3.2.5 Transmitter (Tx) power.....	14
3.2.6 Receiver (Rx) power.....	14
3.2.7 LoRa security.....	16
3.3 LoRaWAN.....	16
3.3.1 LoRaWAN Network coverage.....	17
3.3.2 LoRaWAN Network architecture.....	17
3.3.3 LoRaWAN regions.....	19
3.3.4 LoRaWAN device classes.....	19
3.3.5 LoRaWAN versions.....	19
3.3.6 LoRaWAN Limitations.....	20
3.3.7 LoRaWAN Security.....	21
3.4 Sessions.....	21
3.4.1 Over the air activation (OTAA).....	21
3.4.2 Activation by personalization (ABP).....	23
4 Method.....	25

4.1	Introduction.....	25
4.2	Pre-study.....	25
4.3	Implementation.....	26
4.4	Evaluation.....	27
4.5	Conclusions.....	27
5	Results.....	29
5.1	Introduction.....	29
5.2	Pre-study.....	29
5.3	Implementation.....	31
5.4	Evaluation.....	33
5.5	Conclusions.....	33
6	Discussion.....	35
6.1	Results.....	35
6.2	Method.....	36
6.3	The work in a wider perspective.....	37
7	Conclusions.....	39
	Appendix A.....	41
8	Preparing the hardware.....	41
	Identification.....	41
	System overview.....	42
	General disclaimer and cautions.....	42
	General Raspberry Pi information.....	43
	Dragino LoRa/GPS HAT.....	43
	IMST iC880A-Spi.....	44
	Pycom hardware.....	46
	Arduino Uno.....	46
7.1	Raspberry.....	47
	Adafruit feather.....	48
7.2	Waveshare SX1262 LoRa HAT.....	49
	WaveShare SX1302 LoRaWAN Gateway HAT.....	50
	Appendix B.....	51
9	Installing ChirpStack Gateway OS.....	51

Appendix C.....	55
10 Booting the gateway (Raspberry Pi).....	55
Appendix D.....	59
11 Configuring the gateway.....	59
Appendix E.....	75
12 Connecting a device.....	75
Appendix F.....	77
13 Connecting a Gateway.....	77
7.3 Installing dependencies if you install on top of raspbian.....	79
7.4 Setup the PostgreSQL database and users.....	80
7.5 Installing ChirpStack Network Server.....	80
7.6 Installing ChirpStack Application Server.....	80
Appendix G.....	83
14 Installation on Pycom.....	83
7.7 Hardware setup.....	83
7.8 Install the environment.....	85
7.9 Update the firmware.....	86
Appendix H.....	89
15 Register to TTN/TTS.....	89
Appendix I.....	91
16 Connecting a device.....	91
Adafruit feather.....	91
Bibliography.....	109

List of Figures

Figure 3.1.1: Power consumption vs. range.....	11
Figure 3.3.1: LoRa and LoRaWAN OSI stack.....	17
Figure 3.3.2: LoRaWAN Network architecture.....	18
Figure 7.1: Dragino LoRa/GPS HAT.....	44
Figure 7.2: IMST iC880A, Raspberry Pi 3B+, Tindie backplane, pig tail and antenna.	45
Figure 7.3: IMST iC880A, Raspberry Pi 3B+, Tindie backplane, antenna.....	46
Figure 7.1.1: Arduino project examples. This was just made to get to know Arduino while waiting for the hardware.....	47
Figure 7.1.2: Raspberry examples. 400, Pico with Maker hats, Pi 3B+.....	48
Figure 7.1.3: Adafruit Feather 32u4 with 8.2 cm antenna and a simple temp sensor.	48
Figure 7.2.1: WaveShare SX1262 LoRa HAT.....	49
Figure 7.2.2: WaveShare SX1302 LoRaWAN HAT with GPS.....	50
Figure 7.2.3: Install BalenaEtcher.....	52
Figure 7.2.4: Flash from file.....	53
Figure 7.2.5: IP address in router.....	56
Figure 7.2.6: ssh command.....	56
Figure 7.2.7: ssh -help.....	57
Figure 7.2.8: ChirpStack main menu.....	60
Figure 7.2.9: ChirpStack menu – Select shield.....	61
Figure 7.2.10: ChirpStack menu – GPIO reset pin.....	62
Figure 7.2.11: ChirpStack menu – Concentrator restart.....	63
Figure 7.2.12: ChirpStack - Sub menu 2.....	64
Figure 7.2.13: ChirpStack – General configuration file.....	65
Figure 7.2.14: ChirpStack – help menu.....	66
Figure 7.2.15: ChirpStack – Save.....	66
Figure 7.2.16: ChirpStack – Filename.....	67
Figure 7.2.17: ChirpStack – Concentrator restart.....	67
Figure 7.2.18: ChirpStack – Gateway Bridge config.....	68
Figure 7.2.19: ChirpStack – Gateway Bridge config file.....	69
Figure 7.2.20: ChirpStack – Gateway Bridge config file.....	70
Figure 7.2.21: ChirpStack – MQTT broker address.....	70
Figure 7.2.22: ChirpStack – Enable Node-RED.....	71
Figure 7.2.23: ChirpStack – Configure Wifi.....	72
Figure 7.2.24: ChirpStack – Monit summary.....	72
Figure 7.2.25: ChirpStack – Dashboard.....	78
Figure 7.7.1: Pycom – Hardware.....	84
Figure 7.9.1: Arduino - Settings.....	92
Figure 7.9.2: Arduino – Settings Network.....	93
Figure 7.9.3: Arduino – Tools.....	94
Figure 7.9.4: Arduino – Boards manager.....	95
Figure 7.9.5: Arduino – Boards manager.....	96

Figure 7.9.6: Arduino – Port.....	97
Figure 7.9.7: Arduino – Blink.....	98
Figure 7.9.8: Arduino – Blink 2.....	99
Figure 7.9.9: Arduino – Serial monitor.....	100
Figure 7.9.10: Arduino – AnalogReadSerial.....	101
Figure 7.9.11: Arduino – TempSensor.....	102
Figure 7.9.12: Arduino – Include Library.....	103
Figure 7.9.13: Arduino – Serial monitor.....	103
Figure 7.9.14: Arduino – Serial print.....	105
Figure 7.9.15: Arduino – Library Manager.....	106
Figure 7.9.16: Arduino – TinyLoRa Library.....	107
Figure 7.9.17: Arduino – TinyLoRa Install.....	108

List of Tables

Table 3.1: Typical SNR limits for different Spreading Factors.....	15
Table 7.1: Raspberry Pi pin for LoRa/GPS HAT.....	44
Table 7.2: Raspberry Pi pin for iC880a.....	62

Notation

Abbreviation/ Acronym	Meaning	Explanation	Context
ISY	Institutionen för systemteknik	One of the departments of the Linköping institute of technology. Department of electrical engineering in English.	Department for this thesis.
LoRa	Long Range	Physical layer for wireless modulation	Described in chapter 3.2.
LoRaWAN	Long Range Wide Area Network	LoRaWAN is a MAC Layer Network Protocol	Described in chapter 3.3.
MAC	Media Access Control	A 24 bit (0x123456) address assigned by the manufacturer for the network interface. Used for Gateway ID.	Described in Appendix F.
IoT	Internet Of Things	Physical objects with sensors that process and exchange data in a large network.	Used in chapter 1.1.
TTN	The Things Network	Runs the things stack community that is a LoRaWAN network Server.	Used in chapter 1.4.

Abbreviation/ Acronym	Meaning	Explanation	Context
CSS	Chirp Spread Spectrum	Digital communication that uses wideband linear frequency modulated chirp pulses.	Used in chapter 3.2.
Chirp	A signal that constantly changes its frequency.	A linear increased/decreased frequency pulse	Described in chapter 3.2.
SF	Spreading Factor	Determines the speed of a chirp.	Described in chapter 3.2.2.
BW	Bandwidth	A measure of the width of frequency range.	Used in chapter 3.2.
Hz	Hertz	Unit to measure frequency.	Used in chapter 3.2.
RFID	Radio frequency Identity	A telecommunication standard	Used in chapter 3.1.
ZigBee	Protocol for connecting smart homes or other things.	A telecommunication standard	Used in chapter 3.1.
3G	Third generation mobile phone technology (GSM)	A GSM telecommunication standard	Used in chapter 3.1.

Abbreviation/ Acronym	Meaning	Explanation	Context
4G	Fourth generation mobile phone technology (LTE)	A GSM telecommunication standard	Used in chapter 3.1.
LTE	Long Term Evolution. Standard for mobile broadband over 4G.	A GSM telecommunication standard	Used in chapter 3.1.
OTAA	Over The Air Activation	Dynamic activation method	Described in chapter 3.4.1.
ABP	Activation By Personalization	Static activation method	Described in chapter 3.4.2.
Rx	Receiver		Described in chapter 3.2.7.
Tx	Transmitter		Described in chapter 3.2.6.
TTS	The Things Stack	An open source LoRaWAN Network server.	Mentioned in chapter 3.2.1.
FEC	Forward Error Correction	A method for correcting errors in data transmissions.	Described in chapter 3.2.

Abbreviation/ Acronym	Meaning	Explanation	Context
FSK	Frequency Shift Keying	A modulation method.	Used in chapter 3.2.2.
LPWAN	Low Power Wide Area Network	Wireless communication wide area network designed to allow long range and where the sensors normally are battery powered.	Used in chapter 1.1.
ISM	Industrial Scientific Medical	Radio spectrum reserved internationally for industrial, scientific and medical (ISM) purposes.	Used in chapter 3.1.
ERP	Effective Radiated Power	A standard definition of a directional radio frequency power.	Used in chapter 3.1.
WLAN	Wireless Local Area Network	A wireless computer network standard.	Used in chapter 3.1.
SNR	Signal to Noise Ratio (SNR or S/N)	$SNR = P_{\text{signal}} / P_{\text{noise}}$	Used in table 3.1.

Abbreviation/ Acronym	Meaning	Explanation	Context
NetID	A 24-bit value used for identifying LoRaWAN networks.	If you have a non collaborating network (non roaming) you can use 0x000000 or 0x000001 (free of charge). All other are assigned by LoRa Alliance and you shall not use them if it is not yours. This is 17 bits from the network operator + 7 bits NwkID that also forms part of the DevAddr.	Used in chapter 3.2.1.
APPEUI	A unique 64-bit value assigned when registered at TTN.	If you use a non collaborative network you can use all zeros or a random value but is generated in big endian and written in little endian in the sensor. 0x01, 0x02, ...	Used in sensors for OTAA, see chapter 3.4.1.
DevEUI	A unique 64-bit value assigned by the manufacturer for global identification.	DevEUI is set in the sensor and is normally generated in big endian and written in little endian in the sensor.	Used in sensors for OTAA, see chapter 3.4.1.

Abbreviation/ Acronym	Meaning	Explanation	Context
APPKEY	A 16-byte key used in sensors for OTAA.	This key shall be in big endian format.	Used in chapter 3.4.1.
DevAddr	A 32-bit identifier for identifying end devices within the network.	After joining to the network the communication is done with the DevAddr. (Not unique). ABP uses Static (hardcoded) DevAddr and OTAA uses dynamic.	Used in chapter 3.4.1.
NwkAddr	A 32 bit DevAddr consists of a 7 bits NwkID and a 25 bits NwkAddr.		Used in chapter 3.4.1.

To my cat Soda

Introduction

The need of cheap solutions for various projects within new technologies is increasing with the continuous evolution of internet of things [1]. It is important that new technologies is available to ordinary people without the need of expensive investments, a high technical education and without companies trying to benefit from private data. This could for instance be a need for farmers to set up cheap sensor network to have control over the temperature, food- or water supply and families might also want to have control over the temperature, burglar alarms or snow depth at their summer/winter house without expensive hardware or subscription costs. This could also be used as beneficial for the environment collecting sensor data that could improve knowledge on how to understand the climate and how to avoid and prevent unwanted measured values like in the BRIGHT project.

This thesis gives an introduction to LoRa [2], LoRaWAN [3] and an introduction on how to set up some different affordable solutions using ChirpStack [4]. LoRa means *Long Range* and is the physical layer that handles frequencies and modulation for the above network protocol, on MAC layer, called *LoRaWAN*. ChirpStack [59], [60], [61], is an open-source

LoRaWAN Network Server stack that provides open source components for LoRaWAN networks such as Gateway OS, Gateway Bridge, Network Server and Application Server.

“The ChirpStack open-source LoRaWAN Network Server stack provides open-source components for LoRaWAN networks. Together they form a ready-to-use solution including a user-friendly web-interface for device management and APIs for integration. The modular architecture makes it possible to integrate within existing infrastructures. All components are licensed under the MIT license and can be used for commercial purposes”, see Ref[61]

LoRa means *Long Range* and is the Physical Layer that handles frequencies and modulation for the above network protocol, on MAC Layer, called *LoRaWAN*. ChirpStack is an open-source LoRaWAN Network Server stack that provides open source components for LoRaWAN networks such as Gateway OS, Gateway Bridge, Network Server and Application Server.

This thesis has a lot of appendices that tries to explain how to setup different things.

1.1 Motivation

LoRa/LoRaWAN is one of the fastest growing technologies since it is a complement to other network standards that it integrates to and since it will affect millions of sensors world wide. It enables large scale deployment of Low Power Wide Area Networks (LPWAN) within the Internet of Things (IoT). Through LoRaWAN implementations it is possible to develop various use cases such as solutions for Endangered Species Protection, Smart Agriculture & Animal Production Monitoring, Smart Industry Control, Smart Homes, Smart Cities, Supply Chain Logistics and Smart Metering. Sensor data can further be feed into Machine Learning [5] solutions or combined with other network technologies like in Mesh Networks [6].

1.2 Purpose

The purpose of this work is to describe how to setup a cheap LoRaWAN solution and to evaluate the different setups. The solution should preferably be open source and use common hardware like Raspberry Pi [7]. The main purpose was to setup a LoRa Gateway that also had a Server within the same hardware. The purpose was also to describe how to setup different sensor solutions to communicate with the LoRa Gateway. Further a general description of the environment and standard should be established.

1.3 Problem statements

This thesis mainly considers the following problem descriptions;

- What is needed to setup a cheap LoRaWAN gateway and server solution?
- How can a cheap LoRaWAN system be setup/installed?

1.4 (Research) Limitations

Limitations within this project is mainly due to cost, time, available hardware and the fact that this is not an advanced level thesis.

Since LoRaWAN and IoT is quite a popular technology it has been hard to order hardware. ChirpStack is currently also limited to only support some hardware implementations that limits the scope. I have currently not found any good alternatives to ChirpStack so that limits the scope. The new regulations from TTN, The Things Network [8] that gateways must have at least 2 channels also limits the scope. Further the frequency used for the implementations are limited to EU868 MHz.

Background

2.1 Introduction

This thesis work is part of the BRIGHT project. The BRIGHT project aims to get better knowledges and tools, with focus on heat waves in Swedish cities, and how to better adopt it for climate changes. This thesis will get the reader a brief introduction to further development of LoRaWAN projects mainly with focus on cheaper solutions, mainly with open source and suitable hardware, that can be used for smaller projects whenever needed or for usage in Swedish cities for the BRIGHT project. This thesis is also directed to people that, do not have so much experience in the field of LoRa/LoRaWAN and are interested in new technologies or a have a need of a sensor network where this could be part of an implemented solution.

2.2 Conclusions

After almost a year I can see that it can be quite hard to implement something that appears quite easy. By now I have learnt a lot and spent most of the time

trying to solve bugs on a system that at first appearance would be set up at 30 minutes, if everything would work as supposed. During a year the standards also tend to shift a lot, causing requirements to change. IoT seems to be a fast evolving industry, with a lot of different parameters, causing a lot of changes and a lot of things to learn and get to know. Starting with no experience in the field and no one to ask will be a challenge. This thesis is supposed to facilitate those challenges and give the reader hints for further work in the field.

3.1 Introduction

The purpose of this chapter is to get an introduction to the theory of LoRa and LoRaWAN and get to know related components.

Within this thesis I have studied different hardware and LoRa/LoRaWAN implementations and also some related tools and environments for non professional use, like:

- Arduino UNO [9]
- Raspberry Pi [7]
- Raspberry Pico [7]
- Pycom Lopy4 [10]
- Pycom Pygate [10]
- Adafruit Feather [11]
- Seeed Studio LoRa GPS/HAT [12]

- WaveShare SX1262 LoRa HAT [13]
- IMST GmbH iC880A-spi [14]
- Tindie. IMST iC880a LoRaWAN backplane [15]
- WaveShare SX1302 LoRaWAN Gateway HAT [16]
- 37 Sensor Kit for Uno [21]
- Playknowlogy. Robot car kit. [22]

I have studied books [5], [17] [20], documentation [2], [3], [4], [7], [8], forums [41], online videos [23], [36] and I also took a 3 hp IoT course. A good way to get introduced to LoRa is to watch a video like [23] or [36].

LoRa stands for Long Range and is the physical layer that handles the wireless modulation. LoRa modulation uses different ISM bands and different frequencies. There are also differences regarding single channel gateways and multi channel gateways. Single channel gateways are mainly used in private networks since they are not supported by TTN. They can only work at one frequency and one data rate, see Ref[59].

EU868, EU433, US915 are the most common frequency bands.

LoRa is a complement to other existing wireless communication techniques, see figure 3.1.1, and can be combined with other network standards [6]. There are a lot of different implementations for different telecommunication techniques within different countries, but in general we can say that for Europe:

- LoRa transmits low power small data packets 0.3 to 5.5 kbit/s up to 15 km.
- RFID has a limited range of 1-12m at 2W ERP.
- Bluetooth has a limited range of 1-100m at 100mW.
- ZigBee has a limited range of 1-100m at 100mW.
- WLAN about 150Mbit/s up to 200m and with higher power consumption.

Study of LoRaWAN device and gateway setups

- 3G today has up to 14.4Mbit/s.
- 4G LTE up to 50Mbit/s up to 10km.

The main benefits of LoRa/LoRaWAN are:

- Extreme low power consumption, battery life for years.
- Long range available without licensed spectrum.
- Multi usage, scalable, private or public.
- Cost effective

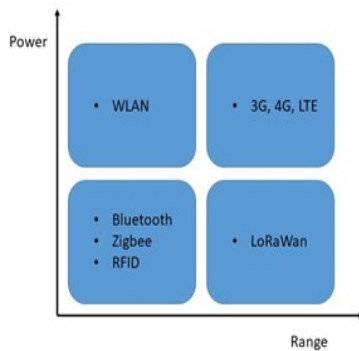


Figure 3.1.1: Power consumption vs. range

3.2 Long-range communication (LoRa)

LoRa uses a proprietary spread spectrum modulation similar to Chirp Spread Spectrum (CSS). A Chirp signal is a signal that constantly changes its frequency. The LoRa modem modulation bit rate, R_b is:

$$R_b = SF \cdot 1 / [2 SF / BW] \quad (1)$$

where SF is the Spreading factor (7 to 12) and BW is the modulation bandwidth in Hz.

LoRa can typically be used for transmitting data between 5km and 15km depending on the obstacles. LoRa allows changing of the transmitter power from 2dBm to 20dBm depending on regulations in different countries that can be useful for saving batteries. LoRa also uses forward error correction (FEC). LoRa is optimized for use with low bandwidth data sent over distances for around 10km and where a low power consumption is required. Typically used for transmitting sensor data once a minute on battery sources like when collection CO₂-, temperature-, or snow data from a city. [24], [25].

3.2.1 LoRa frequencies

There are a lot of different Lora frequencies and LoRaWAN regional parameters.

The main bands are: EU868-870MHz and US902-928 MHz.

For a comprehensive list of frequencies, see reference [27]. There are also some regulations depending on the NetID and usage dependent on membership and now days gateways shall support at least two channels on The Things Network (TTN) that currently is changing to The Things Stack (TTS).

3.2.2 LoRa modulation

LoRa modulation offers trade-off between data rate and sensitivity so that it is possible to optimize the end nodes power. End nodes located close to the gateway should transmit data at a low spreading factor.

The characteristics of LoRa modulation is defined in the LoRaWAN regional parameters [27]. In Europe, the normal band is EU868 that is default 863 to 870 MHz and has 16 channels with 125 kHz bandwidth. LoRa modulation provides almost 50% increased channel capacity compared to FSK [25].

The chirp signal that constantly varies in frequency with the bandwidth same as the signals spectral bandwidth. The payload data signal is chipped at a higher data rate and modulated onto the chirp signal.

LoRa uses 8 up chirps as a preamble, 2 down chirps for synchronisation followed by 5 up chirps with data. The upchirp pulse goes from low frequency to high frequency (/) and the downchirp does the opposite from high frequency to low frequency (\).

Like:

Preamble: “/////” sync: “\\” data: “//\”

3.2.3 Spreading factor (SF)

The Spreading Factor (SF) decides the number of chirps, the data carrier, that are used every second. Normally the Spreading Factor is 7 to 12 depending on the landscape.

Low SF means more Chirps are sent so you can transfer more data and higher SF means more airtime [32] that requires more energy but better sensitivity.

3.2.4 Free Space Path loss

The path loss gives the amount of energy that is lost between the transmitter and the receiver if we assume free space.

$$Path\ Loss(dBm) = 20 \log(distance) + 20 \log(frequency) - 147.55$$

In practise this means that the free space path loss (attenuation) increases when the antennas gets more far away from each other, but it decreases when the wavelength decreases (that is the same as the attenuation increases with the frequency). This also means that the power decreases since the power spreads over an area proportional to the square of the distance.

3.2.5 Transmitter (Tx) power

The transmitter power is the output power of the transmitter.

The output power is regulated and shall not exceed the limits (typically max 20dBm @ 5V). Operating the device outside the specification may result in losing the approval and can damage the device.

Never power up the device without an antenna.

3.2.6 Receiver (Rx) power

The receiver sensitivity is the weakest signal that can be received.

$$Rx\ Power = Tx\ Power - Tx\ connector\ loss + Tx\ antenna\ gain \\ - Path\ loss + Rx\ Antenna\ gain - Rx\ connector\ loss . \quad (2)$$

Rx Power must be > Rx Sensitivity.

This means that the receiver must get a signal with higher power than the receivers sensitivity. When the signal is transmitted with for instance 13 dBm (Rx Power), some energy are lost in the antenna connector cable (assume -1 dBm) and we gain some energy by using an antenna (assume +2 dBm). If the antenna values are the same in the receiver and we know from the specification that the receivers sensitivity is -124 dBm. We can not have a path loss bigger than $124 + 13 - 1 + 2 - 1 + 2 = -139$ dBm. This will be our link budget and with that we can try to optimize our distance if we know the frequency if we assume free space. (In practice Fresnel zones will likely have an effect).

SF	SNR limit
7	-7.5 dB
8	-10 dB
9	-12.5 dB
10	-15 dB
11	-17.5 dB
12	-20 dB

Table 3.1: Typical SNR limits for different Spreading Factors.

Example:

$$Rx\ Sensitivity\ (dBm) = -174 + 10 \log(BW) + SNR\ (dB) + Noise\ Figure\ (dB)$$

$$Link\ Budget\ (dBm) = Tx\ Power\ (dBm) - Rx\ Sensitivity\ (dBm)$$

$$BW = 125\ kHz$$

$$SF = 7 \rightarrow SNR - 7.5\ dB$$

Noise figure = 6 dB

Tx Power = 14 dBm

Rx Sensitivity = $-174 + 10 \log(125000) - 7.5 + 6 \rightarrow$ *Rx sensitivity*: -124 dBm

Link budget = 14 + 124 $\rightarrow$ *Link budget*: 138 dBm

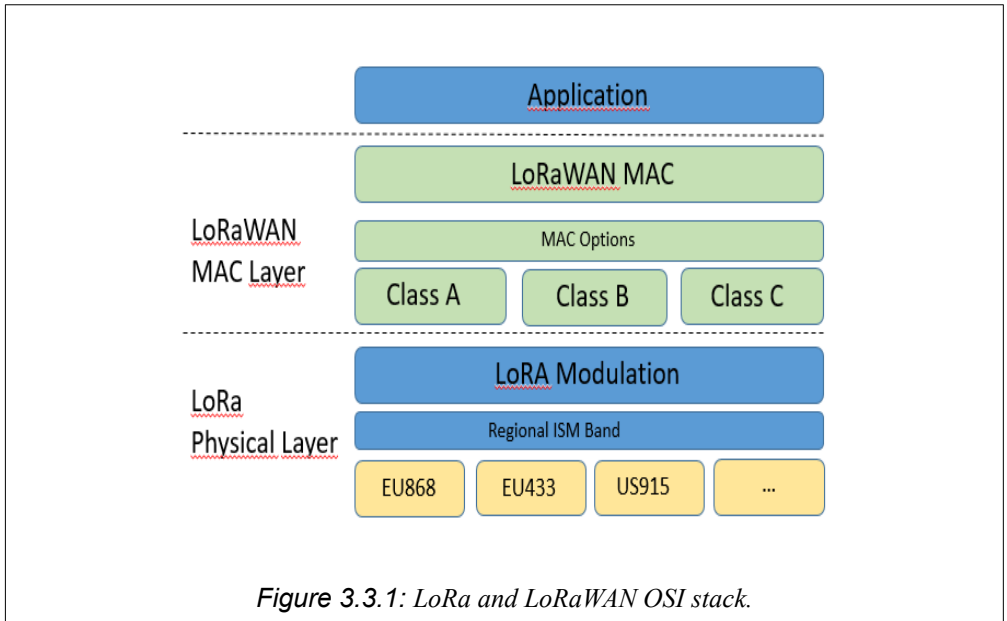
This means that the signal can be attenuated 138 dBm between the transmitter and the receiver.

3.2.7 LoRa security

The physical layer do not provide any security mechanism but it has an optional cyclic redundancy check (CRC) that can be used for error detection.

3.3 LoRaWAN

LoRaWAN is a MAC Layer Network Protocol, see figure 3.3.1.



3.3.1 LoRaWAN Network coverage

The network coverage is very dependent on many things but here are some examples.

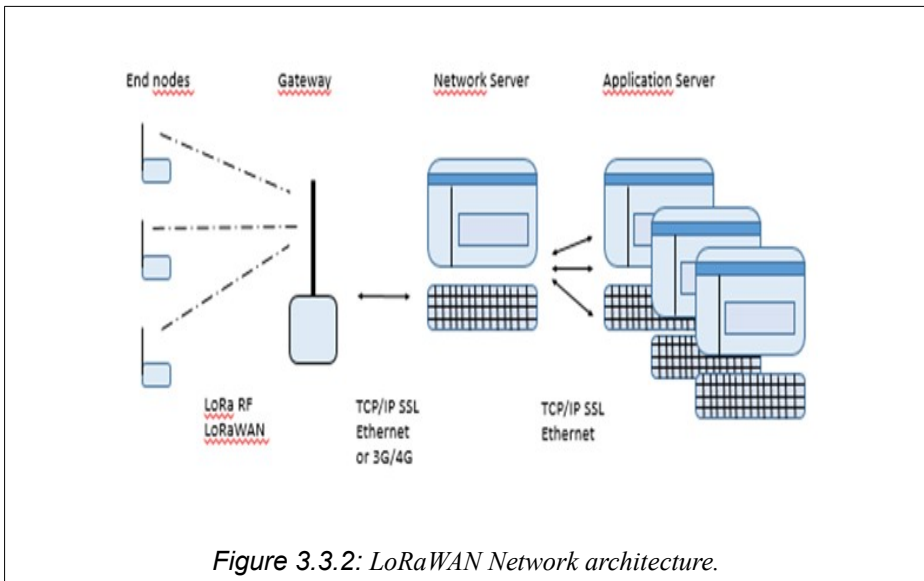
- The theoretical maximum is around 850 km.
- The current world record is 832 km with a balloon.
- Outdoor, high altitude, free line of sight > 10 km.
- Outdoor, house roof, in cities, around 2 km.
- Indoor, around 500m.

3.3.2 LoRaWAN Network architecture

The LoRaWAN network architecture consists of:

Study of LoRaWAN device and gateway setups

- End nodes
- Gateway (Concentrator)
- Network Server
- Application Server
- Geolocation Server



Normally the endnotes is small battery powered units connected to a sensor and sends sensor data on a channel over LoRa/LoRaWAN to the Gateway.

The Gateway receives the sensor data from its sensors and forwards it over Ethernet or WLAN to the Network Server. Normally the Network Server is within The Things Network (TTN) or now-days The Things Stack (TTS) or other Network Servers. Then it is sent back to the operators Application Server.

The main idea of this project is to set up a Gateway that also holds an open source Network Server and Application Server (ChirpStack) on cheaper hardware such as a Raspberry Pi with a suitable HAT.

3.3.3 LoRaWAN regions

See reference [27] for different regional parameters. In Europe, the normal band is EU868 that is default 863 to 870 MHz and has 16 channels with 125 kHz bandwidth. Remember that you must by hardware that supports the correct frequency band otherwise you wont be able to use it.

3.3.4 LoRaWAN device classes

LoRaWAN has 3 different device classes, see Ref[63].

- Class A: All end devices must have Class A. The end device initiates the uplink transmission at any time. When the transmission is completed the end device opens two receive windows (RX1 and RX2) where the network server have to respond.
- Class B: (Beacon) In addition to Class A, the end device can initiate periodically scheduled transmission on fixed interval or after an uplink transmission.
- Class C: In addition to Class A the Class C device keeps the receive window open so that the transmission can initiated by network (a back channel).

In this thesis I will only try Class A devices since the idea is just to send sensor data from the device to the gateway but it is possible to implement all three kinds with Chirpstack.

3.3.5 LoRaWAN versions

There are different versions of LoRaWAN Specifications [33].

- LoRaWAN Specification v 1.0
 - Already withdrawn.
- LoRaWAN Specification v 1.0.1
 - Certification towards 1.0.1 has ended.
- LoRaWAN Specification v 1.0.2
 - First stable version
 - Suits most Class A or C users.
- LoRaWAN Specification v 1.0.3
 - Fully supports unicast and multicast Class B devices.
 - Class A and Class C unchanged compared to 1.0.2 with the exception of a new MAC command.
 - If you use Class B you may need this.
- LoRaWAN Specification v 1.1
 - Adds support for handover roaming, Class B and security enhancements.

The devices I have tested are related to LoRaWAN specification 1.0.2 or 1.0.3. It is quite important that you fill in the correct LoRaWAN specification when you configure Chirpstack.

3.3.6 LoRaWAN Limitations

There are usually some limitations set by different network operators.

- Do not use more than 30s airtime per node per day.
- Do not use more than 10 downlink messages per day.
- Prepaid plans for messages per month.

Private networks shall follow regulations.

In the new regulations on TTN you must have a gateway that supports at least 2 channels. Check your hardware before buying a single channel hardware if you intend to use it as a gateway. If you are planning to use ChirpStack also make sure that your hardware is supported.

3.3.7 LoRaWAN Security

The LoRaWAN layers provides a two-layer AES 128 encryption. A Network Session Key between the End device and the Network Server and an Application Session Key between the End device and the Application Server. The keys are handled different depending on the method used for establish the LoRaWAN session (OTAA or ABP).

The security implementation differs between the LoRaWAN versions and the security implementation matters, like the overall security of the backend, but LoRaWAN is secure by design.

3.4 Sessions

There are two ways of establish a LoRaWAN session.

- Over the air activation (OTAA). – Dynamic.
- Activation by personalization (ABP). - Hardcoded

It is further explained at [54] and [41]. Over the air activation is the most commonly used now days.

3.4.1 Over the air activation (OTAA)

When using OTAA (Over the air activation) the network is dynamically joined. Security keys are negotiated on every activation. The join procedure is performed between the end device and the Network Server and the DevAddr is assigned to the device.

You need a DevEUI that identifies the device globally. That is an 8 byte unique ID assigned by the manufacturer that is linked to the hardware. You normally add 4 zeroes in the middle to what it says on the paper following the device, but it shall also be added in the opposite direction in the device (LSB).

You also need a non unique 8 bytes AppEUI that is 4 byte DevAddr that identifies the device within the network. The DevAddr consists of the end device address within the network (NwkAddr) prefixed by the network identifier (NwkID). For Chirpstack you can use all zeroes or a random value but it should also be entered LSB in the device. You normally get this value by register the device on TTN.

You also need a 16 bytes Appkey that is entered as it was generated in the server.

Join procedure:

1. End device sends a Join Request to the Network Server.
2. The network server lookup the IP-address to the Join Server and sends a Join Request to the Join Server.
3. The Join Server sends a Join Answer to the Network Server.
4. The Network Server sends a Join Accept to the End Device.
5. The End Device sends its data to the Network Server.
6. The Network Server sends the data to the Application Server.

7. The AppKey is delivered.

If you get an “MIC error” on the device data, this likely means that there is something wrong with your keys or its direction. The AppEUI can be set to 0x00, 0x00, ..., 0x00 in Chirpstack if you do not register and get it from TTN.

3.4.2 Activation by personalization (ABP)

When using ABP there is no need for a join procedure. The DevAddr and the security keys are hardcoded in the device.

In Chirpstack you should create a device profile that use ABP and create an application that uses that profile and generate the activation keys. Then you enter those keys in for instance the Arduino ttn-abp example code modified for 32u4 pinning instead of Mo.

Network session key: (0x01, 0x02, 0x03, ..., 0x16)

Application session key: (0x01, 0x02, 0x03, ..., 0x16)

Device address: (0x12345678)

A fixed DevAddr and Session keys for a pre selected network are hardcoded in the end device that can skip the join procedure. For more information see Ref[63].

4

Method

4.1 Introduction

The main idea of this project is to set up a Gateway that also holds an open-source Network Server and Application Server (ChirpStack) on cheaper hardware such as a Raspberry Pi with a suitable HAT and to describe how that is done.

4.2 Pre-study

I started to study the internet for suitable software and hardware that could fit the solution. I also had to read a lot of theory about LoRa and LoRaWAN since it was new to me. I started to evaluate the available hardware, a Raspberry Pi 3B+ [7] with a Seeed Studio LoRa GPS/HAT [12] and some Adafruit Feathers [12] that could be used as end nodes. The Seeed Studio LoRa GPS/HAT only supports a single channel and was not supported by TTN or ChirpStack [4] so I quite fast understood that it would not be a good choice for the project. I quite fast understood that the open source ChirpStack would suite this project perfect.

It seems that IoT and that kind of technology is very popular so I tried to find suitable hardware available to order. It took me almost 2 months to get any hardware so mean while I played around with what I had. I studied Arduino Uno and built a car. I tried Raspberry Pico, some sensors and a lot of other things. When I got the hardware it was not as easy as I thought because there was a lot of things that did not work and the available descriptions could be improved.

4.3 Implementation

In this thesis I will describe:

- How to set up an end node connected to a sensor [12] that transmits data to the Gateway.
- How to set up an 8 channel Gateway with a Pycom LoPy4 and a Pygate with ChirpStack.
- Pycom uses its own Pycom ChirpStack Server implementation so the Gateway just forwards the data to an account on Pycom homepage containing ChirpStack.
- A Raspberry Pi with an IMST iC880A-Spi [14] that is supported by ChirpStack. For this implementation I used a backplane that was ordered from Tindie [15] but it is also possible to just connect it with some wires.
- I also wanted to setup a ChirpStack implementation on a Raspberry Pi with a WaveShare SX1302 LoRaWAN Gateway HAT [16].

I also had an idea that I should make some end nodes from my WaveShare SX1262 LoRa HAT [13] and the Seeed Studio LoRa/GPS HAT on a Raspberry Pi and also use the Raspberry Pico and Arduino for something and finalize the add on kit to my car but that will be out of scope for this thesis.

4.4 Evaluation

The Pycom implementation seems nice and cheap and the hardware was reasonably available. Pycom has some alternatives to the Pygate that might be better if you want to have sensors and evaluate its data. Then a Pysense implementation without ChirpStack could be good since it also supports their AI solution. To install Chirpstack on Pycom and to have to use a login at Pycom homepage and to get a valid key every time you reprogram the unit was maybe not what I wanted to do in the first place. It was positive that Pycom had some other stuff like a suitable IP67 box and antenna kit and that the LoPy4 can be used for other things than LoRa and also can be controlled by an app in the mobile.

The IMST iC880A-SPI seems very popular for the ChirpStack implementation on Raspberry Pi. It is maybe not so good that it does not seem adapted to fit and that you should use the extra backplane that seems to be made by a smaller company and can be hard to get. I guess that you need to put everything in a bigger IP-classed box if you wants to use it outside or wants it to look nice and that could add some cost. Otherwise this fits the requirements with a ChirpStack implementation within the hardware.

Just remember that the Raspberry Pi only can register once on TTN if you are reusing an old Raspberry Pi or you have to try to reclaim the device.

The WaveShare SX1302 LoRaWAN Gateway HAT also seems like a good choice similar to the IMST. It fits better on the Raspberry Pi since it seems to be made for that. It also had a suitable antenna kit and the unit also has a GPS.

4.5 Conclusions

All the 3 above solutions seem reasonable depending on what you want to do and what hardware that is available.

5.1 Introduction

This chapter presents the results and findings of the thesis.

5.2 Pre-study

Pre-studies have been done mainly on internet but I also read some books in different topics. I also attended a training course “*Mid Sweden University*”, called “*Elektronik AV, Effekter och strategier inom IoT, 3hp*”.

There was a lot of things that I needed to study, like:

- Linux: You must know the basics in Linux or Raspbian in order to configure and fault trace Chirpstack.
- Raspberry Pi: You must know the basics of different versions of Raspberry Pi in order to select and configure the hardware.

Study of LoRaWAN device and gateway setups

- **Feathers:** You must know the basics of feather variants in order to understand different libraries, pinning, environment and other related stuff necessary for developing a suitable sensor.
- **Chirpstack:** You need to know the alternatives when installing ChirpStack and what configuration options that is possible to set.
- **TTN:** You need to know the relations to TTN/TTS in order to select hardware and frequencies that are supported and how to register id:s.
- **Programming:** You need to know the basics in programming in order to program devices or raspberry variants. This is mainly c/c++, Python, shell scripts and Java.
- **Arduino:** You need to know how to setup the environment to be able to programme sensors and its libraries and for debugging.
- **Sensors:** You must know the basics of different sensors, what frequencies they have, if they are limited by memory and what pinning they have. It is also good to know the differences of different versions of the “same” sensor.
- **LoRa/LoRaWAN:** You need to know the basics of the LoRa/LoRaWAN standard.
- **Visual Studio:** You need to know the basics of Visual Studio for configuring and programming the LoPy.
- **Electronics:** You need to know the basics of electronics in order to know what pins that should be connected and why.
- **Soldering:** You need to know the basics of soldering since you need to solder antennas and pinnings.

- Network configuration: You need to know the basics of network configuration in order to configure a router and establish a ssh connection to the gateway.
- Telecommunication: You need to know the basics of other telecommunication standards in order to see where LoRa/LoRaWAN comes in to the picture and what it is good for.

The pre-studies have been necessary in order to get to know different technologies used within and around the main focus on the thesis.

5.3 Implementation

The implementations are described in the following appendices:

Appendix A: Preparing the hardware.

This appendix gives a brief description of different hardware that are used and that are not used but could be an alternative. This chapter also gives an overview on how to set up the gateway hardware on a Raspberry Pi with a Tindie card and an IMST iC880a-SPI.

Appendix B: Installing ChirpStack Gateway OS

This appendix instructs how to flash the microSD with Chirpstack.

Appendix C: Booting the gateway (Raspberry Pi)

This appendix gives an overview on how to boot and connect to the Raspberry pi.

Appendix D: Configuring the gateway

This appendix gives a brief overview of Chirpstack menus.

Appendix E: Connecting a device

This appendix instructs how to get started with Arduino and Feather 32u4 for connecting a device. I did not get any data in the server when I tried

OTAA and this might have to do with the 32u4 or the DevEUI(?). With ABP it was quite easy to make it work.

Appendix F: Connecting a Gateway

This appendix gives a brief instruction on how to setup chirpstack-network-server, add a gateway, setup chirpstack-application-server and how to connect a device. The join over OTAA seems to work but I did not get any device data. This could have to do with the device, timing or DevEUI. It worked for ABP so I recommend starting with that if you use an 32u4. I never got the usb driver to work for Mo on Windows 11.

Briefly you shall configure the shield, channel plan, Gateway-ID, check the Mosquitto passwords and set reset pin to 22. Then you need to troubleshoot why it does not work. There is a problem with other reset pins causing the concentrator not to restart so you must run a script and I found out that it was different pins for my two RP3b with identical backplanes and shields. You must also update the `/etc/default/chirpstack-concentrator`. The iC880a uses a sx1301 concentrator channel forwarder and OS full should not have to use MQTT. You must be very careful when you configure the setup the first time because if you do something wrong it will not work. I made both work except the Device Data part and I am currently not sure why. This could also have to do with the timing problems in the sensor or just that the join is rejected because of something strange in the frequency plan. There is a bug about faulty refusing connections due to invalid levels. This appendix also gives brief instructions on how to configure if you install Chirpstack on top of Raspbian.

Appendix G: Installation on Pycom

This appendix describes how to setup a LoPy/Pycom Gateway with Chirpstack. This seems to work but I actually did not verify connecting a device to the Pycoms ChirpStack network server, but I guess it should be the same.

Appendix H: Register to TTN/TTS

A brief instruction on how to register to TTN/TTS. This is actually not within the main scope.

5.4 Evaluation

You need to know a lot of different technologies. This is a bachelor thesis within electrical engineering but the work has mainly been about programming.

At a first glance it seemed easy but there have been challenges. It has been hard to get suitable hardware and the documentation are normally not up to date. Mainly this thesis have been a journey trying to find out, on different forums, why things does not work.

You should preferably select hardware that support the latest standard (TTNv3).

Both hardware, software and standards change over time causing problems. For instance single channel gateways are not supported in TTNv3.

5.5 Conclusions

Industries and technologies within fast evolving products, for a large set of users, using it for many different products are often focused on time to market rather than making a technology that works perfect with all different products and integrations. There will likely be challenges that you have to overcome before you make everything work. I figured that it would be quite easy to setup a LoRaWAN gateway and to connect a device, but today I would say that there can be a lot of traps on the way that has to be solved. It could work and it could not because you do not know about the problems before you have solved them. You should start with the easiest parts and check if that works before going further with other things.

Study of LoRaWAN device and gateway setups

I at least made it work with a Raspberry Pi 3+, Tindie card, ic880A, pig tail, antenna for gateway with Chirpstack servers installed and a Feather 32u4 over ABP. I tried to make the OTAA work for a long time with the 32u4 but it did not get a join accept. This could be because of timing issues or if there was something wrong with the DevEUI in the device (I generated it randomly and did not register it at TTN). For the Feather Mo, I had problems to make the usb driver work in windows 11, so I do not know if it was working at all.

6

Discussion

6.1 Results

The thesis will give a good introduction to IoT, LoRa/LoRaWAN, Chirpstack, environments and related stuff but will not give all the answers. The IoT industry is fast evolving and the implementations will be depending on the hardware you choose and the bugs that exists in the released code depending on the changes in standards, hardware, timing issues and software releases.

The results shows that it is not as easy as it looks to setup this kind of system but I believe the thesis will still be useful for getting to know different solutions and to establish a base knowledge. In the future it might also work without problem.

Gateways:

Lopy4 on Pygate: Set it up with Chirpstack and the gateway seemed to work but I never added any devices in the Chirpstack Network server.

RP3B+ with Tindie, IMST ic880a and Chirpstack OS Full: Worked but had some issues that you need to know about. Tried to set up 2 of them.

Raspberry Pi 4 with WaveShare SX1302: It did not boot so I concentrated on getting the IMST to work instead.

RP3B+ with Dragino LoRa/GPS HAT: This is a single channel HAT and they are not supported, so I figured this would not be suitable for a gateway. Maybe it is more useful as an End Device. I did not evaluate it further.

End Devices:

Arduino 32u4 with OTAA: Never got a join accept to the IMST.

Arduino 32u4 with ABP: Worked directly with the IMST but I did not try ABP until in the end so I have not evaluated it so much.

Arduino Mo: Never got the USB driver to work. Tried some different solutions but Windows did not want to recognize the USB driver. I read that this would be a more suitable solution for OTAA, but anyhow I could not even program the chip.

Arduino MKR WAN: I did not have time to evaluate it but it seems as a good option.

Waveshare sx1263 on Arduino UNO: Did not have time to try it.

6.2 Method

The method has been incremental elaborative.

First I have made pre-studies and then I have focused on getting to fulfil the objective increment. This thesis have been more of a practical elaboration than a theoretical study but of course the elaboration requires some theory.

6.3 The work in a wider perspective

The work in a wider perspective tries to solve the problem on how to, practical set up an instruction, for how to configure a cheap platform that can be used for the collecting measured data and report it to a server. The main idea is to collect and send temperature data from many sensors to a gateway and being able to read out values over internet to make a contribution to the BRIGHT project. Actually so that you do not need to bicycle around and read out the values over Bluetooth.

Conclusions

This thesis has investigated different network solutions for how to setup networks that can be used for transferring sensor data. The focus has been to setup a solution that has a gateway with a built in network server and an application server. Third party software have been used for the setup. The results did not lead all the way to the goal. The setup of the gateway (RP3+/ic880a) worked but it was big problems to receive the sensor data from the node correctly. This is likely due to timing problems and the chosen connection type for the used sensor hardware but could also depend on other things like problems with the chosen software libraries or if the DevEUI is faulty configured. Finally it worked when I tried ABP instead.

Appendix A

Preparing the hardware

This appendix gives a brief instruction on how to setup some hardware and just shows examples of some other.

This document mainly describes how to setup a ChirpStack LoRaWAN® Network Server using a Raspberry Pi and a LoRa HAT

There are also other useful instructions and descriptions for other sources, for instance in Ref[37] and [38].

Identification

This document identifies the following hardware:

- General Raspberry Pi
- IMST iC880A-Spi
- Tindie backplane
- Pig Tail
- Antenna

Study of LoRaWAN device and gateway setups

- Power Supply
- Dragino LoRa/GPS HAT
- IMST iC880A-Spi
- Pycom (Appendix G)
- Arduino Uno
- Raspberry Pi
- Adafruit Feather
- Waveshare SX1262
- Waveshare sx1302

System overview

A node with a chosen sensor, for instance a temperature sensor, sends data over LoRa, to a gateway. The gateway receives and acknowledges the connection and forwards the data to a network server. The network server forwards the data to an application server.

General disclaimer and cautions

DISCLAIMER: This instruction is available for usage as is.

The writer of this thesis will not be held responsible for any usage of this instruction causing economic or other problems.

CAUTION: Electronic circuits are sensitive to electrostatic discharges (ESD), make sure to handle it appropriate.

CAUTION: Connect the antenna to the LoRa connector before powering the Raspberry Pi or it will break.

CAUTION: Read the manufacturers carefully before using the hardware.

CAUTION: Follow the frequency regulations.

General Raspberry Pi information

- Connect the screen to the HDMI port
- Note: Raspberry Pi 4 uses micro HDMI.
- Connect the keyboard and a mouse to the USB ports
- Connect a network cable to the network interface. It will use DHCP as default.
- It is also possible to use WLAN
- Connect the flashed microSD Card to the MicroSD Card reader.
- Connect the power cable to the micro USB (USB-B).
- Note: (Raspberry Pi 4 uses USB-C). If you does not use a Raspberry Pi power supply that can deliver 3A@5.1V, you might get problems that the screen complains or similar depending on the load.
- Raspberry Pi model A+/B+ and 4 uses 40 pin I/O the older ones have 26 I/O ports.
- Older models also use a SD card instead of a micro.
- The micro SD card should be at least 16 Gbyte.
- In Raspberry Pi 1B+, 2B, 3B, 3B+ the microSD card slot is compatible with maximum 32GB (if you does not use FAT) so do not buy a to big microSD card.

Dragino LoRa/GPS HAT

Since this Dragino is a single channel Gateway [41], that has limits on TTN and is not supported by ChirpStack, so I will not write a description about it. There are useful guides at mobilefish, see [35] or in the Seeed Wiki [39] on how to connect to the Loriot Server [40] or at the Dragino wiki[41].



Figure 7.1: Dragino LoRa/GPS HAT

Raspberry Pi	LoRa/GPS HAT
Pin 4	DIO1
Pin 5	DIO2

Table 7.1: Raspberry Pi pin for LoRa/GPS HAT.

Connect the LoRa/GPS HAT to the Raspberry Pi. (Compatible with Raspberry Pi 2 Model B or Raspberry Pi 3 Model B/B+)

IMST iC880A-Spi

Connect the iC880A-Spi [14] to the Tindie backplane [15]. If you do not have a backplane, you can connect it with wires according to the documentation in the iC880A-Spi Quick start guide [58]. Connect the backplane and the 868MHz antenna to the Raspberry Pi, see Figure 7.2 and Figure 7.3.



Figure 7.2: IMST iC880A, Raspberry Pi 3B+, Tindie backplane, pig tail and antenna.

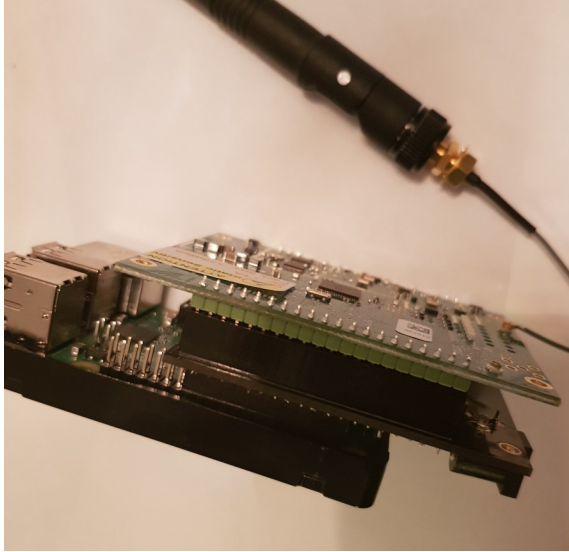


Figure 7.3: IMST iC880A, Raspberry Pi 3B+, Tindie backplane, antenna.

Pycom hardware

The Pycom hardware is described in Appendix G.

Arduino Uno

It is good to know something about the Arduino. It uses c/c++ instead of python and is smaller than the Raspberry Pi, but is good for smaller project. The Arduino has a lot of examples and add on hardware, if you want to build other things that does not require a Raspberry Pi. For instance I tried a car and a display project while studying this. See Figure 7.5.

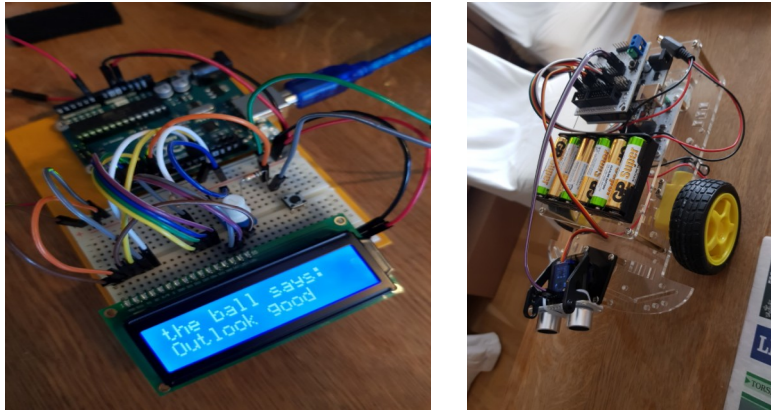


Figure 7.1.1: Arduino project examples. This was just made to get to know Arduino while waiting for the hardware.

7.1 Raspberry

The Raspberry family also has different hardware. The raspberry Pico is good for smaller projects. There are a lot of different Raspberry's and it is useful to know about those. I tried some variants like the Raspberry Pico, Raspberry Pi 3B+, Raspberry Pi 4, Raspberry Pi 400. The 400 is built into a keyboard but is otherwise similar to a raspberry Pi 4.

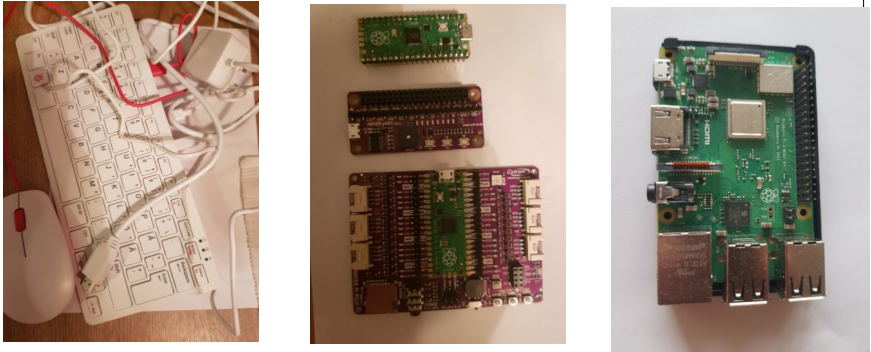


Figure 7.1.2: Raspberry examples. 400, Pico with Maker hats, Pi 3B+

Adafruit feather

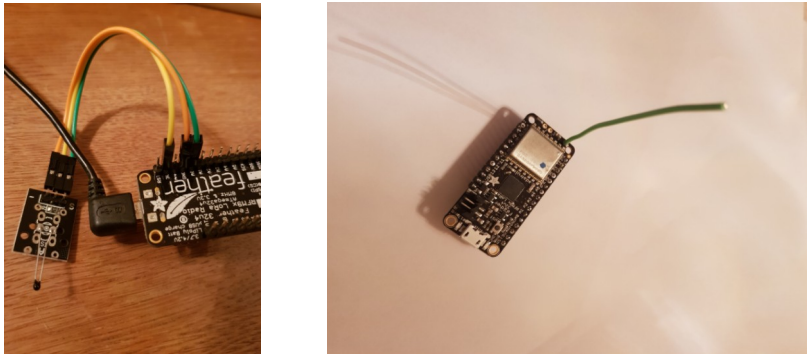


Figure 7.1.3: Adafruit Feather 32u4 with 8.2 cm antenna and a simple temp sensor.

The Adafruit feather comes in different variants. I have tried the 32u4 but maybe the MO is a little more useful, mainly if you want to use OTAA.

I have described it further in Appendix I. It is useful as a LoRaWAN node with simple sensors. Arduino MKR WAN is also a good alternative for sensors. I only got the Feather 32u4 to work with ABP.

7.2 Waveshare SX1262 LoRa HAT

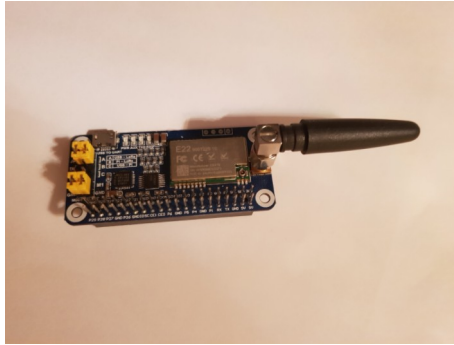


Figure 7.2.1: WaveShare SX1262 LoRa HAT

I have not had the time to test the Waveshare yet but I planned to use it on an Arduino Uno if I could find out a sensor project that requires some more hardware resources.

WaveShare SX1302 LoRaWAN Gateway HAT



Figure 7.2.2: WaveShare SX1302 LoRaWAN HAT with GPS

I have not had the time to try the SX1302, but it seems like a good alternative to the IMST iC880A-SPI for use together with ChirpStack.

Appendix B

Installing ChirpStack Gateway OS

This appendix describes how to install a suitable image on a micro-SD card.

1. Download a suitable image from ChirpStack [56].
2. Make sure you select the "chirpstack-gateway-os-full" image for the suitable Raspberry Pi version.
3. It is suggested that you use the Balena Ripley tool to flash the card, see Ref[57].
4. Use a suitable adapter, see Figure , to mount the micro-SD card to the computer.
5. Flash the card.

After you have installed the micro SD card and booted your gateway, before you unplug the power cord of your gateway you should turn it off first. You can do that with "sudo shutdown -h now" otherwise the files might get corrupt.

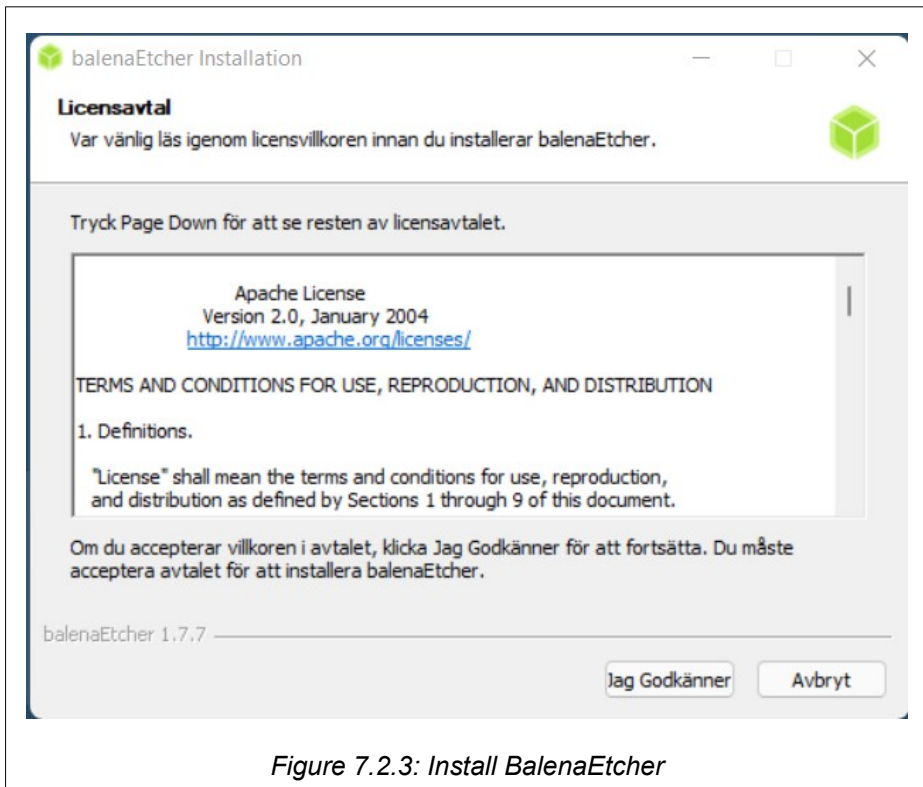
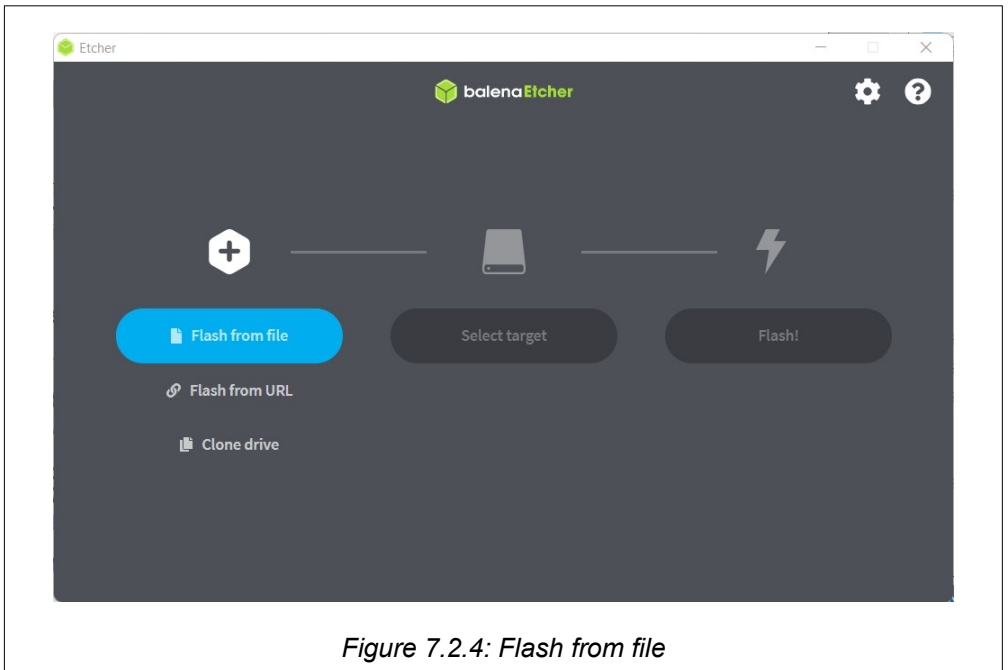


Figure 7.2.3: Install BalenaEtcher



Appendix C

Booting the gateway (Raspberry Pi)

This appendix describes how to setup the chirpstack-gateway-os-full image on the first boot and how to connect to the Raspberry Pi.

1. Insert the prepared micro-SD card into the Raspberry Pi.
2. Connect a keyboard, mouse, monitor, network cable and a Raspberry Pi power supply. (or just the power supply if you connect from another computer).
3. The Chirpstack-gateway-os-full image will setup the PostgreSQL on the first boot.
4. After booting the gateway (Raspberry Pi), you will need to login using Secure Shell (ssh). The IP address for the gateway (if you used an Ethernet cable) will use DHCP, so you must first find out what address it has. (It will not be possible to login the normal way after you have rebooted the gateway). It is normally also possible to connect wireless to Chirpstack_AP (192.168.0.1) but on some version you must enable it first. (ssh [admin@192.168.0.1](ssh://admin@192.168.0.1) with default pwd admin).

Study of LoRaWAN device and gateway setups

5. It is normally easiest to log in to your router and check the connected items to get the IP address:

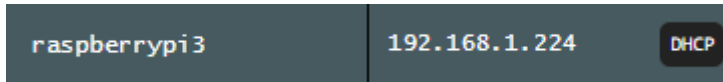


Figure 7.2.5: IP address in router.

6. On Windows, open a command window (search cmd) and write “`ssh admin@192.168.1.224`”. You must do this on a computer that is connected to the same network.

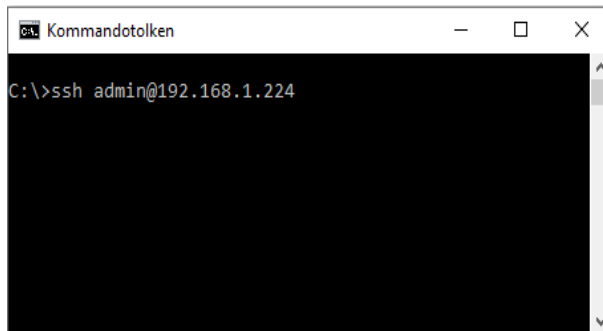


Figure 7.2.6: ssh command

If that did not work try `ssh -help`

```
C:\>ssh -help
unknown option -- h
usage: ssh [-46AaCfGgKkMnqsTtVvXxYy] [-B bind_interface]
          [-b bind_address] [-c cipher_spec] [-D [bind_address:]port]
          [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11]
          [-i identity_file] [-J [user@]host[:port]] [-L address]
          [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port]
          [-Q query_option] [-R address] [-S ctl_path] [-W host:port]
          [-w local_tun[:remote_tun]] destination [command]

C:\>
```

Figure 7.2.7: ssh -help

Then try again...

You should get a question that you are supposed to answer yes to (the first time) if you want to connect.

At the first connect you normally have to copy the key to a file in [c:\>\users>\<user>\ .ssh\known_hosts](#) and in the .txt file add like (create the .ssh folder and the text file if it does not exist)

```
192.168.1.8 SHA256:lsfZOd7WhFLO/DdAmdSs2nNpoKEZarcSp5VxdWYV4Es.
```

7. The default password is “admin”. Remember to change the default password (use menu option 8 in the next appendix or “passwd” after you logged in to change the default password).

Appendix D

Configuring the gateway

This appendix describes the menus and an introduction on howto configure the chirpstack-gateway-os-full image.

1. Type: “`sudo gateway-config`” in the command window (cmd in windows). Enter the password again when asked to do so.

2. The ChirpStack–gateway-os-full currently only supports the following

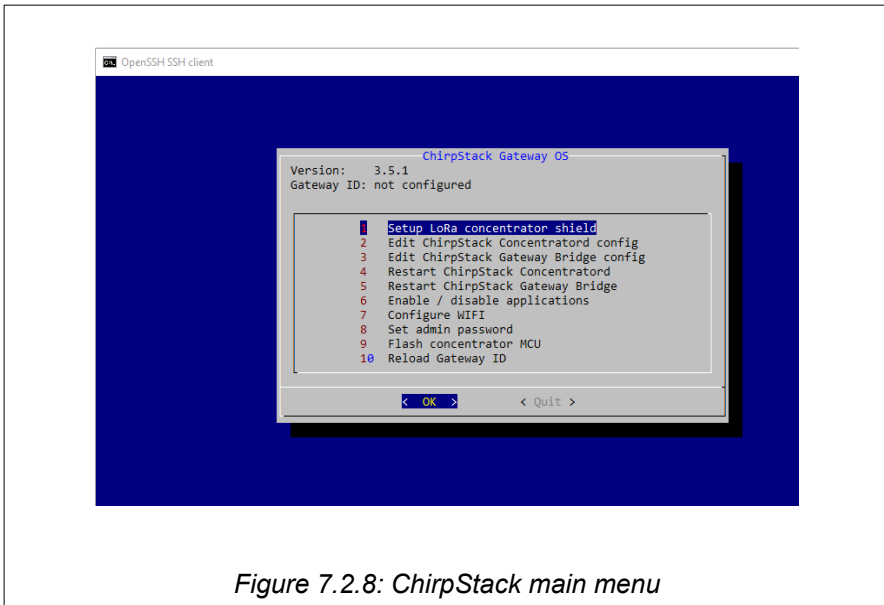


Figure 7.2.8: ChirpStack main menu

shields on Raspberry Pi:

- IMST – iC880A
- IMST – iC980A
- IMST- Lite Gateway
- Pi Supply – LoRa Gateway Hat
- RAK – RAK2245
- RAK – RAK2246
- RAK2246G (with GNSS)
- RAK2287 (with GNSS)
- RAK – RAK831 (Gateway Developer Kit)
- RisingHF – RHF0M301 (LoRaWAN IoT Discovery Kit)
- Sandbox Electronics – LoRaGo PORT
- Semtech – SX1280 (2.4 GHz)
- Semtech – SX1302 CoreCell

Menu 1: Setup LoRa concentrator shield (use arrows to move up/down), see Figure 7.11.

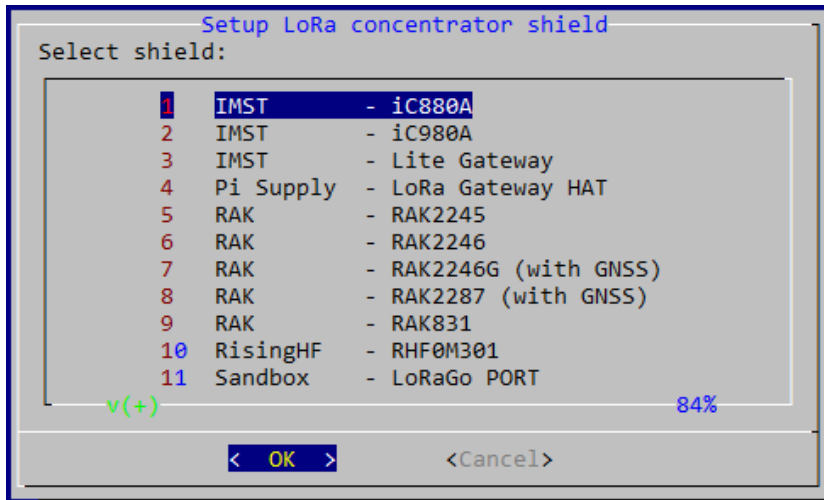


Figure 7.2.9: ChirpStack menu – Select shield

3. Choose selected shield.
4. Choose frequency (EU868MHz)
5. Fill in GPIO pin when it asks for it:

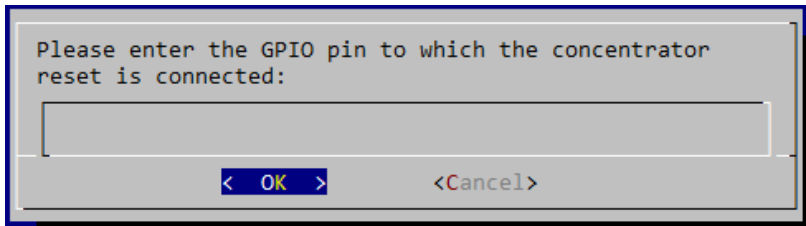


Figure 7.2.10: ChirpStack menu – GPIO reset pin

Raspberry Pi physical pin	iC880a pin	Description
2	21	5V Supply
6	22	Ground (GND)
19	16	Master Out Slave In (MOSI)
21	15	Master In Slave Out (MISO)
22	13	Reset
23	14	Serial Peripheral Interface (SPI) clock
24	17	Negative Slave Select (NSS) – Means active low

Table 7.2: Raspberry Pi pin for iC880a.

6. This should normally mean **pin 22** for the IMST iC880a but you can change it in the General configuration file.

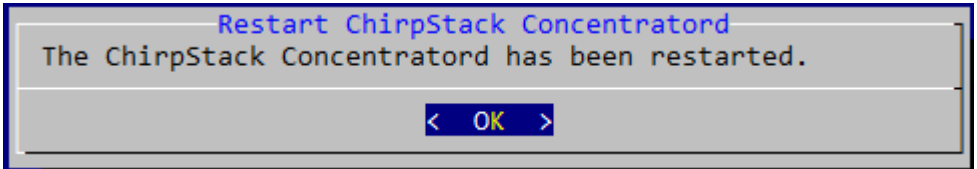


Figure 7.2.11: ChirpStack menu – Concentratord restart

Menu 2: Edit ChirpStack Concentratord config

Menu 2.1: General configuration

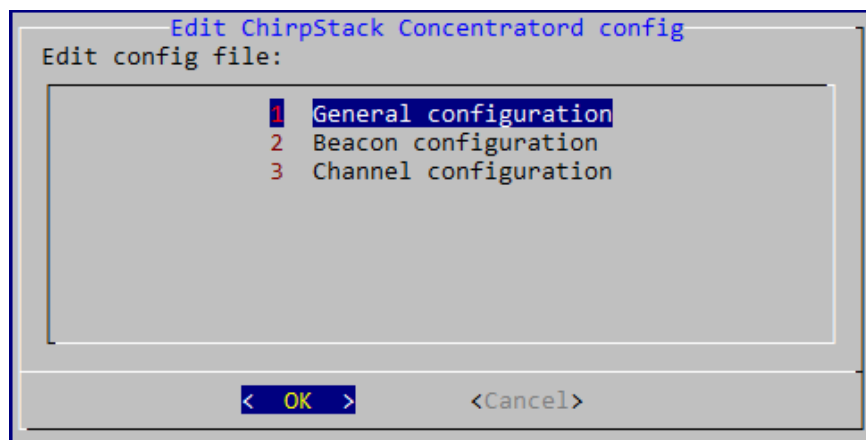


Figure 7.2.12: ChirpStack - Sub menu 2

General configuration file.

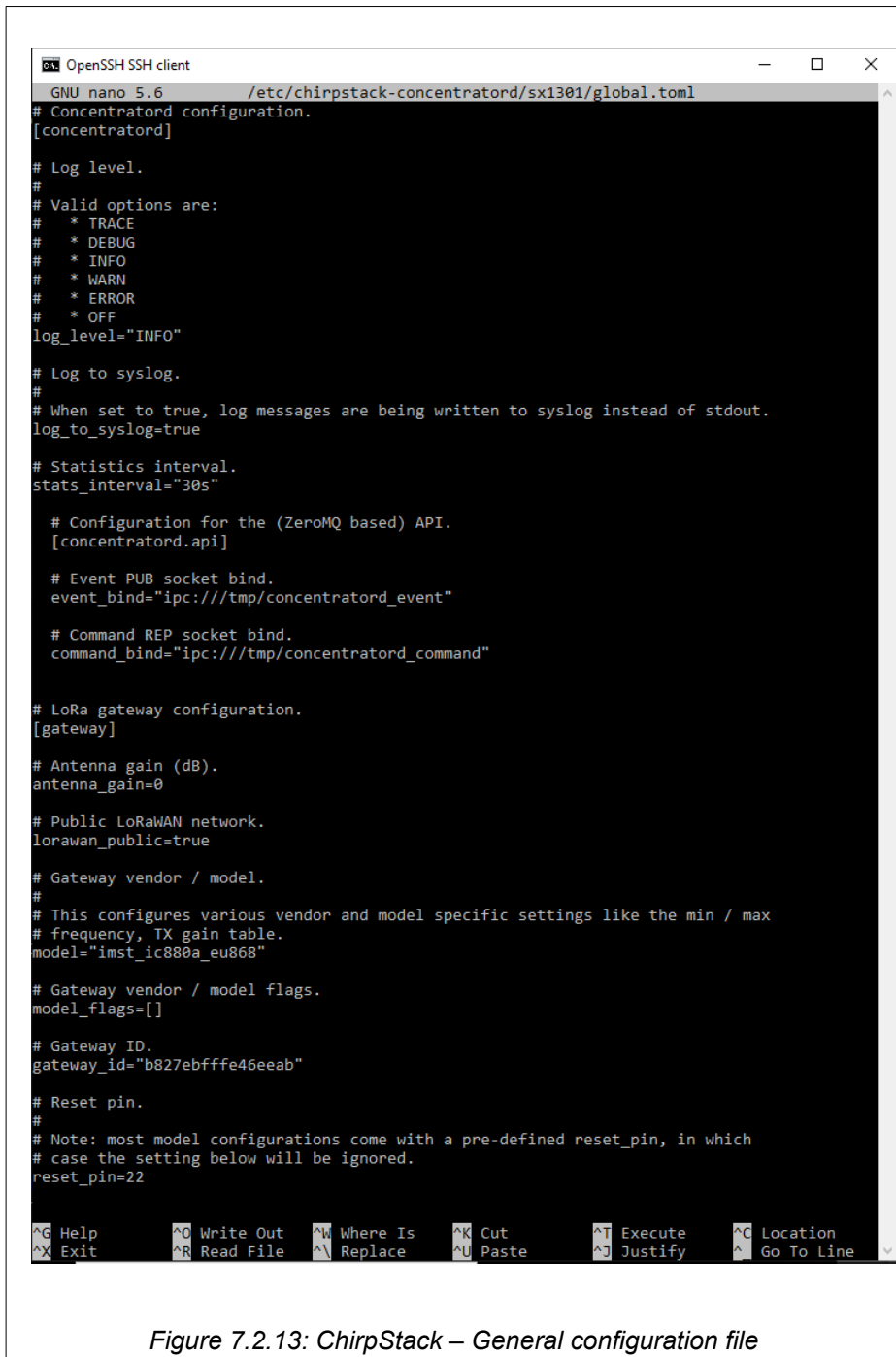
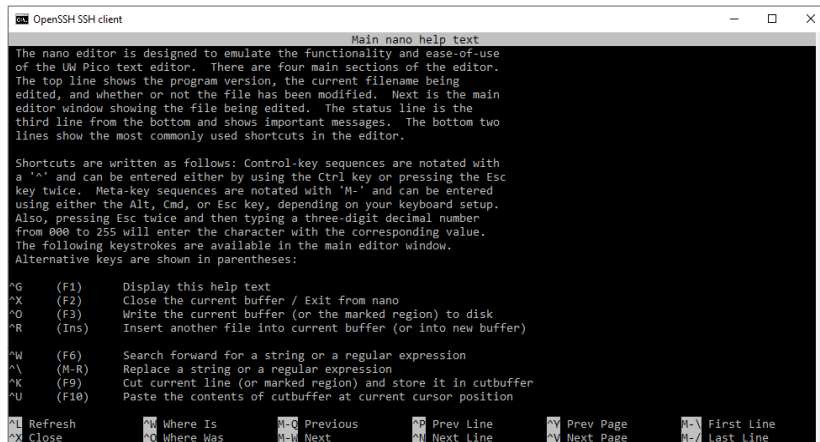


Figure 7.2.13: ChirpStack – General configuration file

Study of LoRaWAN device and gateway setups

^G (CTRL+G) – Shows the help menu.



```
OpenSSH SSH client

Main nano help text
The nano editor is designed to emulate the functionality and ease-of-use
of the UW Pico text editor. There are four main sections of the editor.
The top line shows the program version, the current filename being
edited, and whether or not the file has been modified. Next is the main
editor window showing the file being edited. The status line is the
third line from the bottom and shows important messages. The bottom two
lines show the most commonly used shortcuts in the editor.

Shortcuts are written as follows: Control-key sequences are notated with
a '^' and can be entered either by using the Ctrl key or pressing the Esc
key twice. Meta-key sequences are notated with 'M-' and can be entered
using either the Alt, Cmd, or Esc key, depending on your keyboard setup.
Also, pressing Esc twice and then typing a three-digit decimal number
from 000 to 255 will enter the character with the corresponding value.
The following keystrokes are available in the main editor window.
Alternative keys are shown in parentheses:

^G (F1)    Display this help text
^X (F2)    Close the current buffer / Exit from nano
^O (F3)    Write the current buffer (or the marked region) to disk
^R (Ins)   Insert another file into current buffer (or into new buffer)

^W (F6)    Search forward for a string or a regular expression
^_ (M-R)   Replace a string or a regular expression
^K (F9)    Cut current line (or marked region) and store it in cutbuffer
^U (F10)   Paste the contents of cutbuffer at current cursor position

[Alt] Refresh      [Alt] Where Is      [Alt] Previous      [Alt] Prev Line
[Alt] Close        [Alt] Where Was   [Alt] Next         [Alt] Next Line
[Alt]              [Alt]              [Alt]              [Alt] Next Page
[Alt]              [Alt]              [Alt]              [Alt] First Line
[Alt]              [Alt]              [Alt]              [Alt] Last Line
```

Figure 7.2.14: ChirpStack – help menu

7. Change the file as you want it to be. I will start to use it as a private network and I have a 2dBi antenna so I changed:

- lorawan_public=false

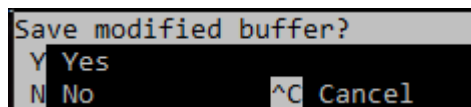
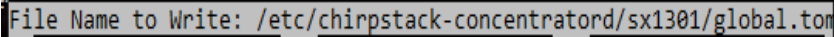


Figure 7.2.15: ChirpStack – Save

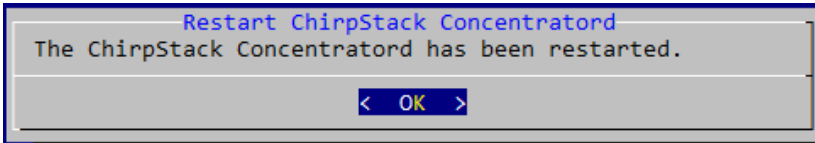
When finished press CTRL+X and answer Y for yes to save the changes, and then press return to accept the filename.



```
File Name to Write: /etc/chirpstack-concentrator/sx1301/global.toml
```

Figure 7.2.16: ChirpStack – Filename

The ChirpStack Concentrator will restart.



```
Restart ChirpStack Concentrator
The ChirpStack Concentrator has been restarted.
< OK >
```

Figure 7.2.17: ChirpStack – Concentrator restart

8. Press OK or return to get back to the menu.

- Menu 2.2: Beacon configuration
- Not included in this document.
- Menu 2.3: Channel configuration
- Not included in this document.

Menu 3: Edit ChirpStack Gateway Bridge config

Sub menu 3.0:

Menu 3.1: Edit config file

Study of LoRaWAN device and gateway setups

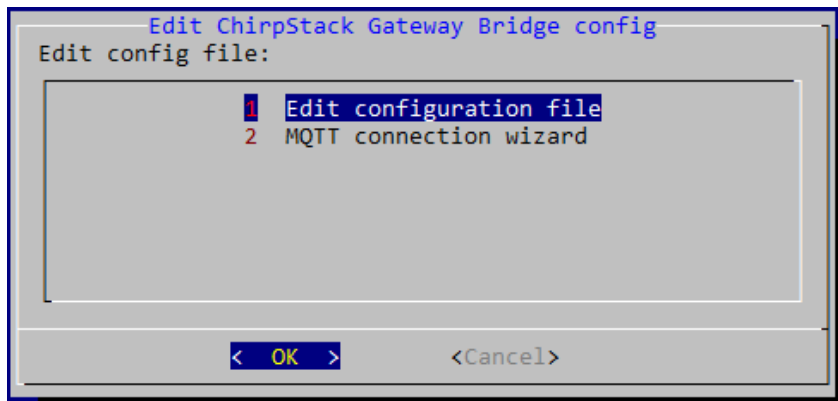


Figure 7.2.18: ChirpStack – Gateway Bridge config

You should not need to change this file, but if you want to find out more you can check the example at reference[55].

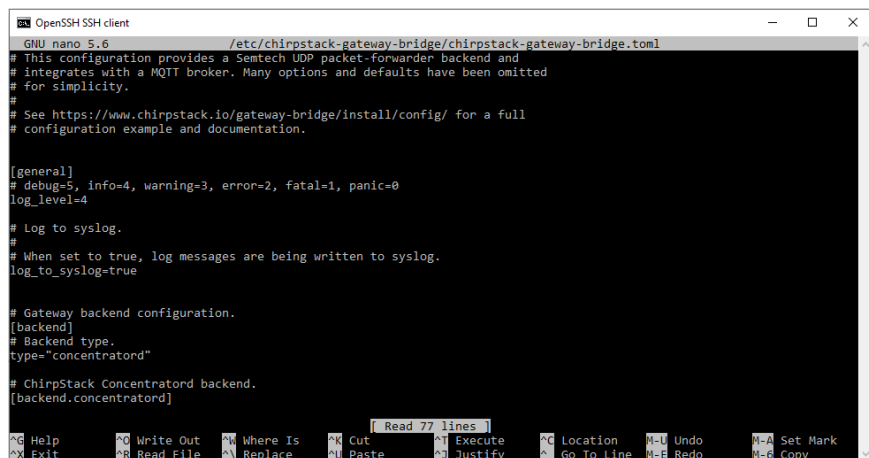


Figure 7.2.19: ChirpStack – Gateway Bridge config file

Study of LoRaWAN device and gateway setups

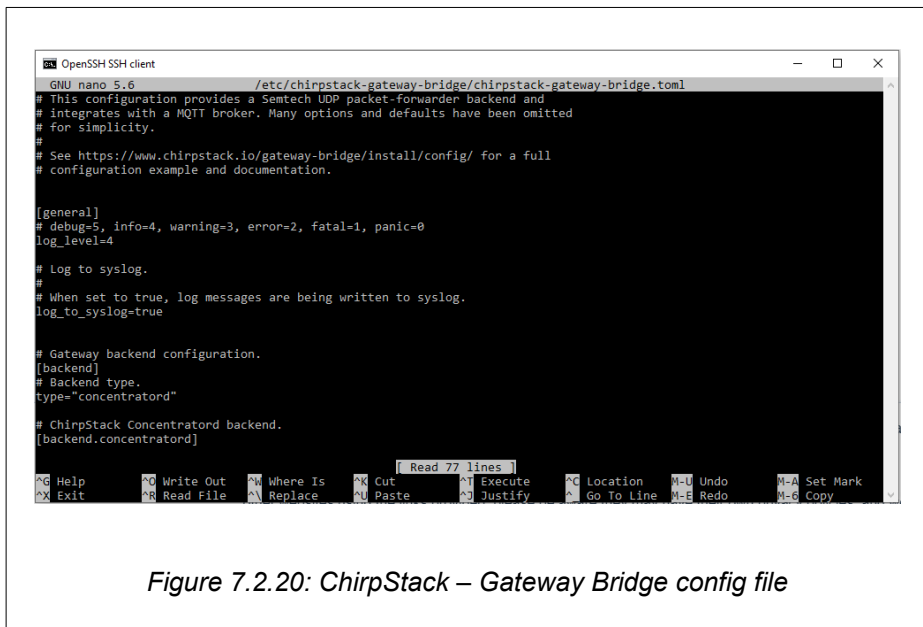


Figure 7.2.20: ChirpStack – Gateway Bridge config file

Menu 3.2: MQTT connection wizard

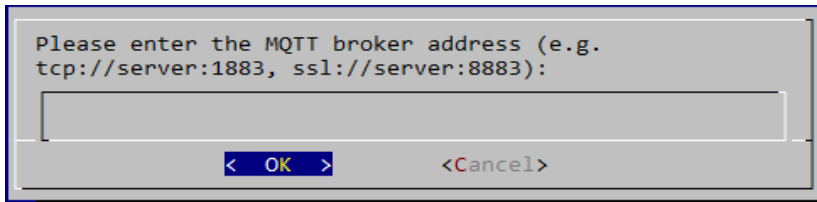


Figure 7.2.21: ChirpStack – MQTT broker address

This should be the same as you have set in the config-file above (IP 127.0.0.1 for loopback address when you have it installed on the same Raspberry Pi).

```
server="tcp://127.0.0.1:1883"
```

- Menu 4: Restart ChirpStack Concentrator
- Menu 5: Restart ChirpStack Gateway Bridge
- Menu 6: Enable / disable applications

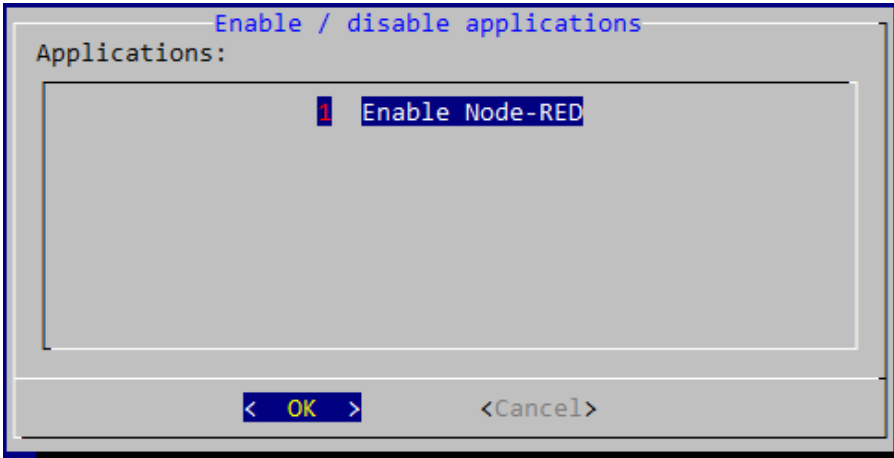


Figure 7.2.22: ChirpStack – Enable Node-RED

You normally do not need to enable Node-RED.

Node-RED provides a browser based flow editor to wire together flows of nodes. Once enabled you can access it on your gateways address at port 1880. It takes some time to enable it and you should configure the server to localhost port 1883. If it is enabled this menu will change to disable Node-RED. Read more about it in [4].

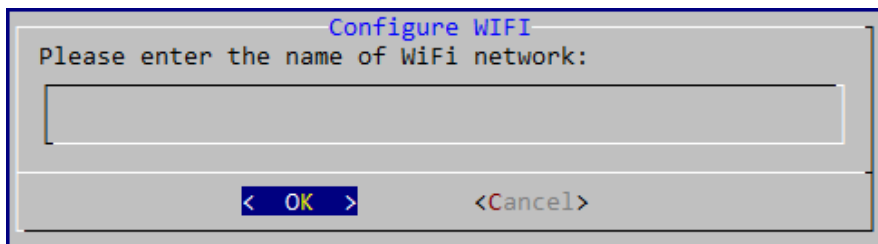


Figure 7.2.23: ChirpStack – Configure Wifi

Study of LoRaWAN device and gateway setups

- Menu 9: Flash concentrator MCU
- Menu 10: Reload Gateway ID

```
raspberrypi3:~$ sudo monit summary
Password:
Monit 5.27.2 uptime: 6d 22h 29m
```

Service Name	Status	Type
raspberrypi3	OK	System
chirpstack-network-server	OK	Process
chirpstack-gateway-bridge	OK	Process
chirpstack-concentrator	OK	Process
chirpstack-application-server	OK	Process

```
raspberrypi3:~$
```

Figure 7.2.24: ChirpStack – Monit summary

Exit back to the prompt and check if everything is working.

Sudo monit summary

There have been some issues with chirpstack-concentrator “*does not exist*”. Depending on different hardware versions the reset pin can be different, meaning the concentrator does not get a reset and therefore do not start.

If you have this problem, with IMST iC880a, you can try to change the file: “/etc/default/chirpstack-concentrator” and add:

```
SX1301_RESET_BCM_PIN=5

# reset pin (5 or 25)
CONCENTRATOR_RESET_PIN=5

# power enable pin
CONCENTRATOR_POWER_EN_PIN=18

# concentrator version
```

Study of LoRaWAN device and gateway setups

```
CONCENTRATOR_VERSION="sx1301"
```

```
# reset the LoRa concentrator on Concentrator start
CONCENTRATOR_RESET="yes"
```

and then run the following script as root (sudo su):

```
#!/bin/bash

# Reset iC880a PIN
SX1301_RESET_BCM_PIN=5
echo "$SX1301_RESET_BCM_PIN" > /sys/class/gpio/export
echo "out" >
/sys/class/gpio/gpio$SX1301_RESET_BCM_PIN/direction
echo "0" >
/sys/class/gpio/gpio$SX1301_RESET_BCM_PIN/value
sleep 0.1
echo "1" >
/sys/class/gpio/gpio$SX1301_RESET_BCM_PIN/value
sleep 0.1
echo "0" >
/sys/class/gpio/gpio$SX1301_RESET_BCM_PIN/value
sleep 0.1
echo "$SX1301_RESET_BCM_PIN" > /sys/class/gpio/unexport
```

In one of my gateways it worked with `SX1301_RESET_BCM_PIN=5` and in the other `SX1301_RESET_BCM_PIN=25` but the reset pin in gateway-config shall still be 22.

You can also verify that SPI is enabled.

```
"sudo nano /boot/config.txt" and verify that
"dtparam=spi=on"
```

Appendix E

Connecting a device

When you are using the “chirpstack-gateway-os-full” image, you should not update the ChirpStack Gateway Bridge configuration. It is advised to have a ChirpStack Gateway Bridge on each gateway itself, to enable a secure connection between the gateway and server. When using the “chirpstack-gateway-os-full” image, the “ChirpStack Gateway Bridge” is configured to point to the MQTT broker that follows the installation. It just points to the loopback address and the assigned port, see Ref[59].

This means the mqtt address is set to “servers=[“tcp://127.0.0.1:1883”,] on local host, no ssl (scheme://host:port).

The “ChirpStack Gateway Bridge” acts as a backend for the packet forwarder.

The packet forwarder depends on the vendor and model, and ChirpStack supports three different package forwarders:

- ChirpStack Concentratord
- Semtech UDP Packet Forwarder
- Semtech BasicStation

The packet forwarder forwards data to the ChirpStack Gateway Bridge and also controls the LoRa chipset. It must be configured for the correct frequencies and it must match the band and channels in the ChirpStack Network Server Configuration.

After you have configured the channel plan, you shall continue with chapter 8. Configuring a gateway and chapter 9. Connecting a device.

Appendix F

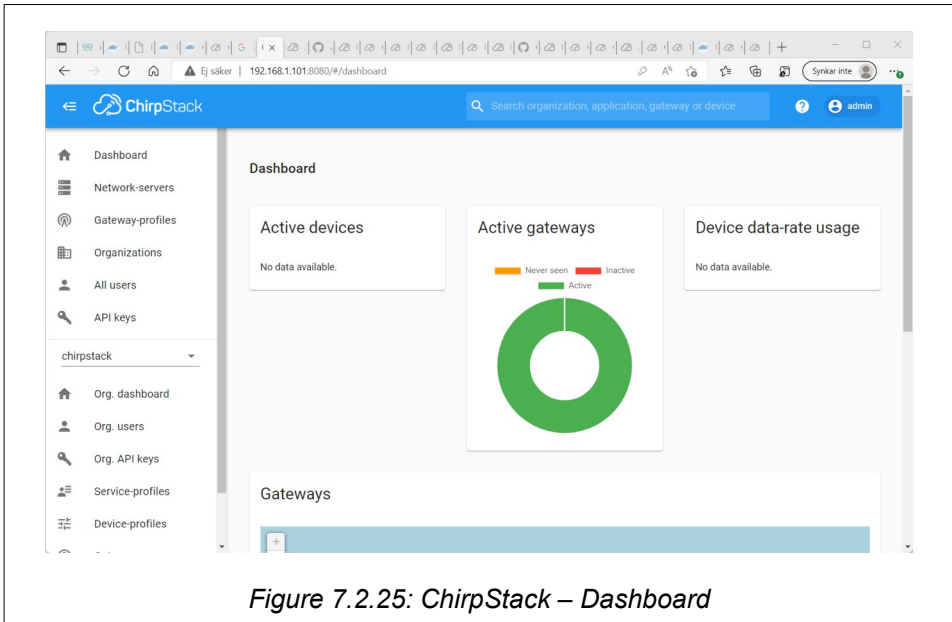
Connecting a Gateway

To add a gateway you shall login to the ChirpStack Application Server web-interface with default:

- Username: admin
- Password: admin

You connect to the web interface by typing the IP-address:8080 of your gateway (Raspberry Pi) in the address field of a browser after you have it up and running and have connected it to the ChirpStack Network Server and created/connected a Service profile of your organization with the network-server.

Navigate to Gateways in the web-interface and + Create, and complete the form. The Gateway ID must be equal to the Gateway ID in your physical Gateway otherwise it will not work. This is normally the MAC address with fffe in the middle.



1. Goto Network-servers, Add, give the Network-server a name and set the address to localhost:8000.
2. Click on the Link on the Network-server if you want to enable gateway discover or generate certificates, but this should not be necessary for the functionality.
3. Click on Gateway-profiles, create. Give it a name, set the Stats interval and enable the channels. The channels shall comply with the configuration of the gateway. Do not forget to click update gateway-profile.
4. Click on Service-Profiles, create. Give it a name, enable gateway meta-data, and if you did not registered on TTN/TTS set it to Private gateways. Set the min/max allowed datarate to 0/5 for Eu.
5. Click on Gateways, create. Fill in the same gateway ID as you have set up in your gateway (Registered ID or MAC + fffe in the middle). Add

altitude and coordinates if you want. Under gateway configuration, give it a name, description, select the service profile and gateway-profiles. Update Gateway.

6. Goto Device-profiles. Give it a name. Set the LoRaWAN MAC version (1.0.2?) and LoRaWAN Regional Parameters revision (B?), Default ADR algorithm Max EIRP (14?) and Uplink interval (30?). Goto JOIN (OTAA/ABP), Enable “Device supports OTAA” or not (ABP). Goto Codec and at least select “none”. Update Device-profile.

7. Goto applications. Create. Give it a name and a device profile. Generate a Device EUI. In the configuration give it a name, description and a device profile. Disable frame-counter validation. Generate a key under Keys (OTAA) if you are using OTAA. If you are using ABP generate Device address, Network Session Key and Application session Key. Update device.

7.3 Installing dependencies if you install on top of raspbian

This chapter is mainly hints if you install ChirpStack manually on top of raspbian (raspberry pi OS) or a Debian operating system.

The following are assumed to have been installed:

- MQTT broker
- Redis
- PostgreSQL

Otherwise you can install them with:

```
sudo apt install mosquito mosquito-clients redis-server redis-tools postgresql.
```


7.4 Setup the PostgreSQL database and users

Start the command line utility for PostgreSQL:

```
sudo -u postgres psql
```

Use this prompt to set up the databases that ChirpStack uses and change the passwords:

You shall set the same values in chirpstack-network-server and chirpstack-application-server.

```
create role ChirpStack_as with login password 'dbpassword';  
create role ChirpStack_ns with login password 'dbpassword';  
create database ChirpStack_as with owner 'dbpassword';  
create database ChirpStack_ns with owner 'dbpassword';
```

7.5 Installing ChirpStack Network Server

Install the package with apt:

```
sudo apt install chirpstack-network-server
```

You will find the configuration file at:

```
/etc/chirpstack-network-server/chirpstack-network-  
server.toml
```

You must edit the file so that it matches the database configuration and the correct frequency settings.

7.6 Installing ChirpStack Application Server

Install the package with apt:

Study of LoRaWAN device and gateway setups

```
sudo apt install chirpstack-application-server
```

You will find the configuration file at:

```
/etc/chirpstack-application-server/ChirpStack-  
application-server.toml
```

You must edit the file so that it matches the database configuration.

```
[postgresql]  
  
dsn="postgres://ChirpStack_as:dbpassword@localhost/  
ChirpStack_as?sslmode=disable"
```

That is the ChirpStack_as must have the correct location (localhost) and password

```
[application_server.external_api]  
  
jwt_secret="verysecret"
```

You shall select a secure password of your own.

After you have saved the file you can start the chirpstack-application-server:

```
sudo systemctl start chirpstack-application-server  
  
Start chirpstack-application-server on boot:  
  
sudo systemctl enable chirpstack-application-server
```

(On ChirpStack OS you use “monit” instead of “systemctl”).

Print the logs from the ChirpStack Application Server:

```
sudo journalctl -f -n 100 -u chirpstack-application-server
```

Appendix G

Installation on Pycom

This guide will describe how to setup ChirpStack on a LoRaWAN 8-channel Gateway on Pygate and Lopy4 hardware. In order to do this you must register the device on TTN and create an account on Pycom. There is also a similar instruction in Ref[47] or Ref[10].

7.7 Hardware setup

To set up a Pygate and Lopy4, to be used as an 8-channel LoRaWAN Gateway, connect the Lopy4 on the Pygate according to picture.



Figure 7.7.1: Pycom – Hardware

The RGB LED, the push button and the two antenna connectors shall be towards the USB power connector on the Pygate. You must connect the LoRa antenna before using the output otherwise it might break. Connect the LoRa antenna to the connector on the left side of the USB power connector on the Pygate. You do not have to use the antennas on the Lopy4. For additional information or a more complete instruction, see [10]. Note the warnings if you intend to use PoE-NI. There is also a shorter video in Ref[36].

7.8 Install the environment

You could start with the firmware upgrade but I believe it might work better in this order at least if you use wifi (and have no PPOE since that is not assumed here) since you must configure the network (wifi) settings at Pycoms homepage and in the device (JSON) in order to make the registration work and you also get the registration key that you shall use, when doing the firmware upgrade, at the homepage and after about 20 minutes you must generate a new one if it took to long time.

You can use Atom [43] or VS Code [44] to connect to the device.

I tried VS Code on Windows 11, so I will only try to explain that.

Download and install Visual Studio from [45].

Download and install, the long term support version, of Node.js from Ref[46]. Install this before Pymakr.

Connect a USB-C to USB cable between the Pygate and the computer (a data USB cable, not just a power cable).

Start VS Code (with the board connected or not).

Install the Pymakr VSCode Extension to enable communication with the Pycom board.

In VS Code, on the Extension tab, search and install Pymakr.

Note: You can verify that VS Code is in the path by open a Command Prompt (cmd) and write “code” and press enter.

Connect the Pygate with the USB cable and on the Pymakr console you should get a “>>>” prompt. Otherwise you might have a faulty (non data) USB Cable or

it might be connected to the wrong USB port or you might have some other program using the USB resource?

At the “>>>” prompt it shall be possible to program the device with microPython. If you do not know anything about microPython there is an introduction in Ref[48] or since Raspberry Pi and Pico also uses MicroPython there are also some guides at [7].

7.9 Update the firmware

To flash the Pycom Device with a firmware build where Pygate functionality is enabled, download the appropriate tool recommended by Pygate, see [34] and install it on your computer.

Create an account at Pycom and login. If you already have a Webshop account you can use that.

- In the Pycom main configuration window select “add device”.
- Add device via USB.
- Select LOPY4, from “Choose your device - all menu”.
- Then choose PYGATE, from “Choose your device - shields menu”.
- Goto PYGATE settings menu:
 - Choose “Provider: Pycom Chirpstack Server”.
 - LoRa Region: EU868.
 - Click next.

When you select ChirpStack you will get a login emailed to your to your registered email address, so that you can login to the Pycom ChirpStack Servers, here only provided through a Pycom login.

If you do not have a PPoE device for Ethernet, you must enable wifi.

- Add device – Enable networks.
 - Choose Wifi.
 - Click next.

Study of LoRaWAN device and gateway setups

- Add a New Wifi
 - Fill in SSID: <Network_Name>
 - Choose: Network secured with password
 - Enter the password
 - Click “Save”.

After you have enabled wifi you likely must change the wifi settings in your json-file so that the device can connect to the correct network.

If you do not have a connection your device wont be able to register the code you flash and it will not forward the data to the server.

- In “Add a device – Customize your device”, you shall now have got a name, wifi settings and the correct devices shown (Pygate and Lopy4).
- Press save.

Goto your newly created device and select “Provisioning”.

- Choose: Offline firmware updater.
- Download the firmware updater.
 - Choose operating system for the downloader (Windows).
- Connect the device to the USB of your computer.
- Notice the “Activation Token”, that you only can use for 20 minutes, otherwise you can/must generate a new one.
- Flash Pybytes firmware with Pycom firmware updater tool.
- Connect to the ChirpStackAP if possible or you can do that later.
- Follow the instructions:
 - Welcome menu:
 - You do not need to check development releases or send anonymous data.
 - Click: “Continue”.
 - Make sure the serial port is not used by any other tool.
 - Communication menu:
 - Choose serial port.
 - Check: “High speed transfer”

Study of LoRaWAN device and gateway setups

- Select type: “Pybytes”.
 - Check: “Force update pybytes registration.”
 - Check: “Enable Pybytes / SmartConfig support.”
 - Click: “Continue”.
- In “Pycom upgrade – Device type”.
 - Choose board Lopy4.
 - Click: Continue.
- In “Pycom upgrade – LoRa region selection”.
 - Choose: <Country>
 - Choose: <Region> (EU868).
 - Click: “Continue”.
- In “Pycom upgrade – Sigfox region selection”.
 - Sigfox region “Europe”.
 - Click: Continue.
- In “Pycom upgrade – Pybytes registration”.
 - Enter the Activation Token. (It is only valid for 20 minutes otherwise you must generate a new one).
 - Click: Continue.

Now it shall start to upgrade. (Writing Lopy.bin at 0x00....)

After a while you shall get a result screen “Your Lopy4 was successfully updated”...

- Click: Done

Appendix H

Register to TTN/TTS

In order to get everything to work you must create an account on TTN [8]. Actually you will be redirected to the newer TTS[54]. Press “Sign up” and enter Username, e-mail address and a password.

You will have to verify the e-mail address and then you can login.

There is already good guides at TTS[54] so I won't write so much about it.

It can be good to know that the Gateway EUI, also called DevEUI, is a 64 bit unique ID assigned by the manufacturer. It should be equal to the MAC address of your Raspberry Pi with an extra fffe in the middle. It's normally easiest to find it in your router, and you shall simply fill it in without semicolon “;” or colon “:”. You can also see it as “Gateway ID” in the file when you do the ChirpStack general configuration. In the device you shall set it in the opposite order, 0x08, 0x07, ...

Appendix I

Connecting a device

Adafruit feather

The Adafruit feather 32u4 has 3.3V logic and is clocked at 8 MHz.

You can power it with a 3.7V Lithium battery, that has a built in battery charger. There is a similar guide at reference [51] if you need more information that is not included here.

- Solder the header strips layout that is suitable for your solution.
 - Normally 16+12 plain male headers.
- Solder a wire antenna.
 - For 868MHz it shall be 82 mm.
 - Optional you can solder an antenna connector instead.

Study of LoRaWAN device and gateway setups

- Power with a 5V Micro USB (and/or with a 3.7V rechargeable battery).
 - It can be tricky to remove the battery, without destroying the connector, when you have inserted it, so I suggest that you wait with the battery.
- Arduino IDE Download
 - Start the IDE and navigate to file/preferences:

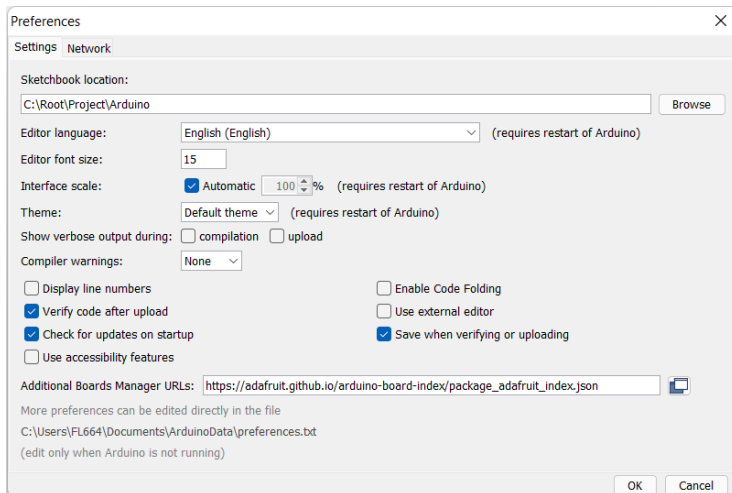


Figure 7.9.1: Arduino - Settings

Your Sketchbook folder should normally be named “Arduino”, on windows, make a “libraries” folder, for libraries, in it if it does not exist (do not give it a random name).

Make sure to add the “Additional Boards Manager URLs”

Study of LoRaWAN device and gateway setups

[https://adafruit.github.io/arduino-board-index/
package_adafruit_index.json](https://adafruit.github.io/arduino-board-index/package_adafruit_index.json)

In the "Network"-tab set your proxy settings or "Auto-detect proxy settings".

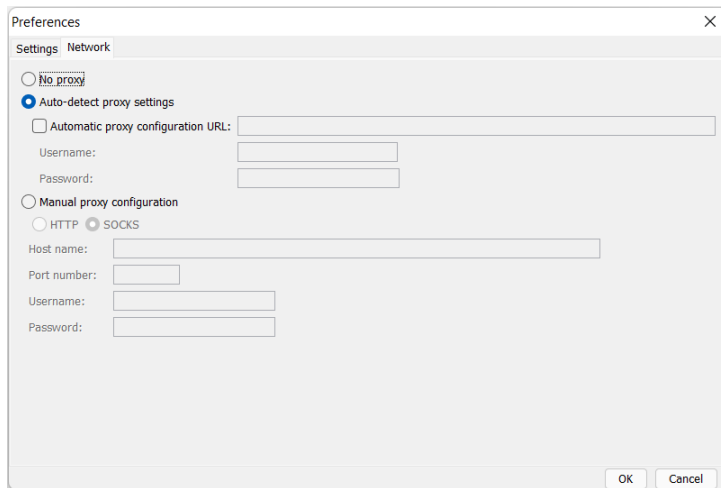


Figure 7.9.2: Arduino – Settings Network

In "Tools => Boards Manager" install the Adafruit AVR Boards

Study of LoRaWAN device and gateway setups

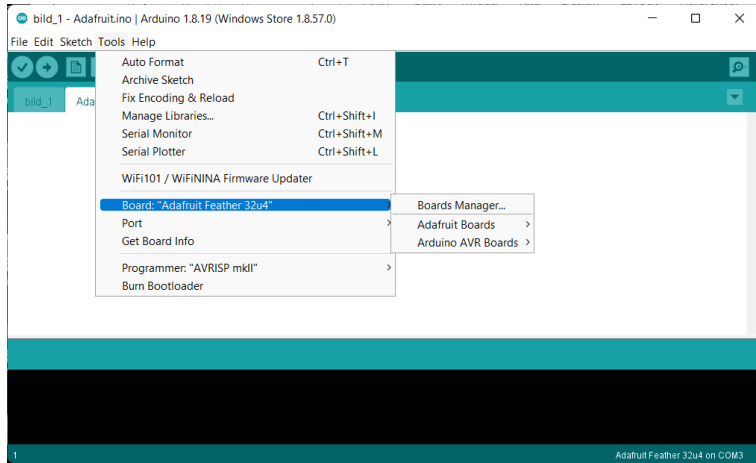


Figure 7.9.3: Arduino – Tools

Select the correct board In:

“Tools => Board” => Adafruit Boards => Adafruit Feather 32u4

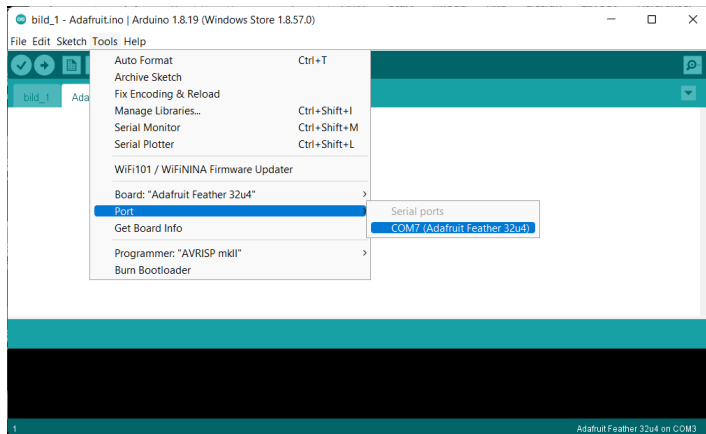


Figure 7.9.6: Arduino – Port

Select COM port.

Make sure you have a Micro USB cable that supports data and is not only for powering.

Normally the port will recognize the connected unit and write it out...

Port: “COM5 (“Adafruit Feather 32u4”)

Open an example to test the device: file => examples => 01.Basics => Blink

Study of LoRaWAN device and gateway setups

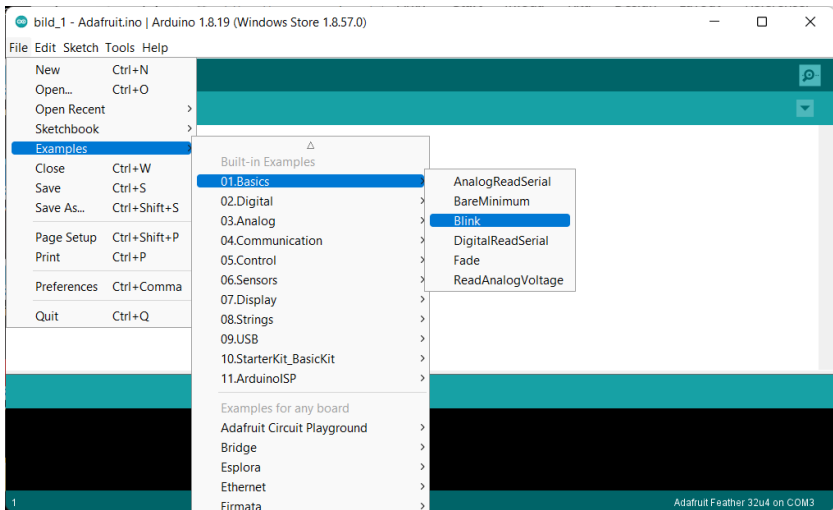


Figure 7.9.7: Arduino – Blink

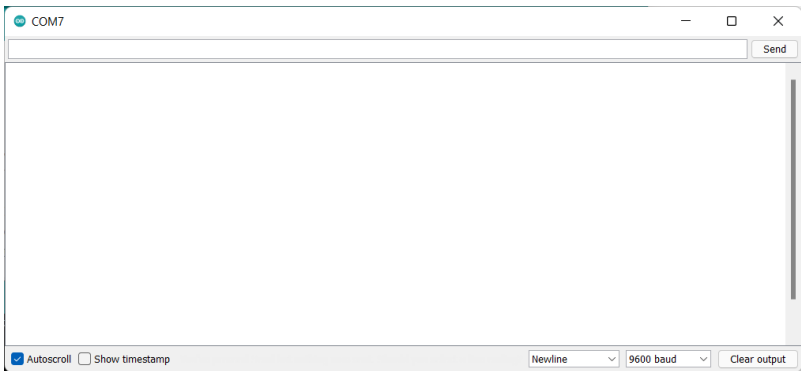
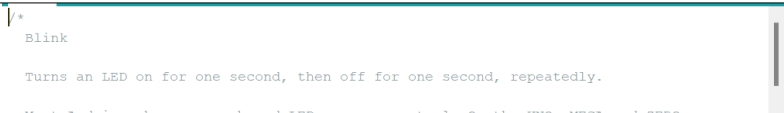


Figure 7.9.9: Arduino – Serial monitor

Study of LoRaWAN device and gateway setups

Now we are ready to make some more tests with a sensor.

Connect your sensor to the Adafruit Feather.

We are using a simple temperature sensor.

Connect the sensor to Ao, 3.3V and GND

Open the AnalogReadSerial example code and modify it so it reads from Ao.

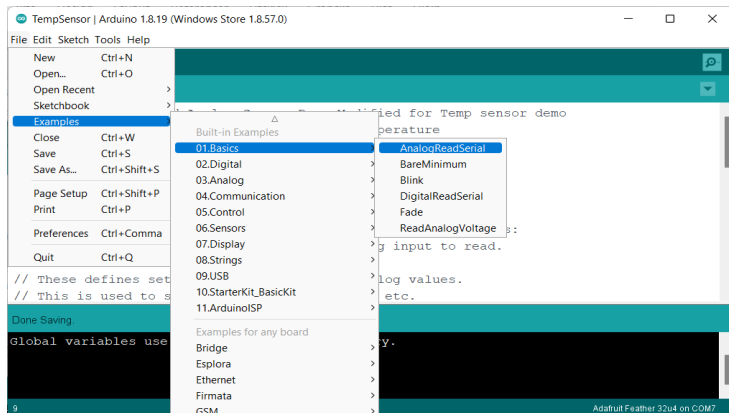


Figure 7.9.10: Arduino – AnalogReadSerial

Study of LoRaWAN device and gateway setups

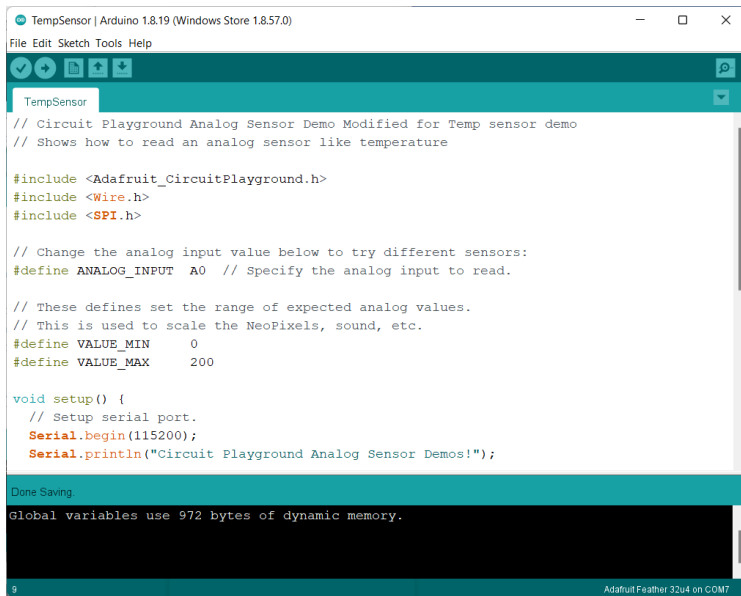


Figure 7.9.11: Arduino – TempSensor

If you try to compile it will likely ask for the libraries?

Save the modified file in your “Arduino” folder and it should create a folder with the same name as the file. You can import missing libraries from,

Sketch => Include Library.

When it compiles you can download it to the Adafruit Feather and you should be able to see some digits in the Serial Monitor.

You can add code to calibrate the sensor if you like.

The sensor is typical linear. Actually it is using a Steinhart-Hart-Equation. see Ref[52] and [53], but for now we assume it is linear.

$$y = k \cdot x + m$$

Where “x” is the temperature and “y” is the measured voltage.

If you heat the sensor with your fingers, you can see if it goes up or down.

From the first measure x was about 22 degrees giving a value of 535.

I put it outside where it is -2 degrees and got 790.

This gives somewhere near $y = -10x + 770$

You can probably calibrate it better, but then you can modify the code;

```
uint16_t value = analogRead(ANALOG_INPUT);  
  
uint16_t temperature = (value - 770)/-10;  
  
Serial.println(temperature, DEC);
```

Notice that this is a 16 bit unsigned integer meaning it shall be positive (0..65535) and avoid zero division (if value = 770).

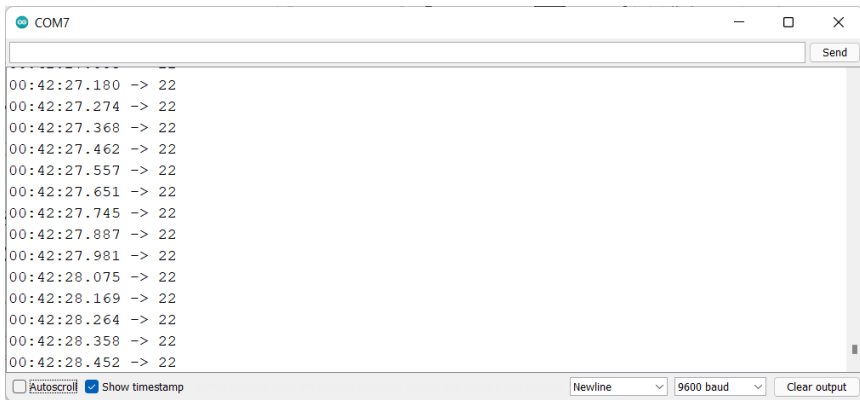


Figure 7.9.14: Arduino – Serial print

Another way to install libraries on Windows is;

Close the Arduino IDE since libraries only load at startup.

Goto [Adafruit/Adafruit-GFX-Library](#) on Github [49].

Read the guide about it or some other interesting projects at Adafruit learning system [50].

For the LoRaWAN sensor project you need to install the “MCCI LoRaWAN LMIC library”.

Study of LoRaWAN device and gateway setups

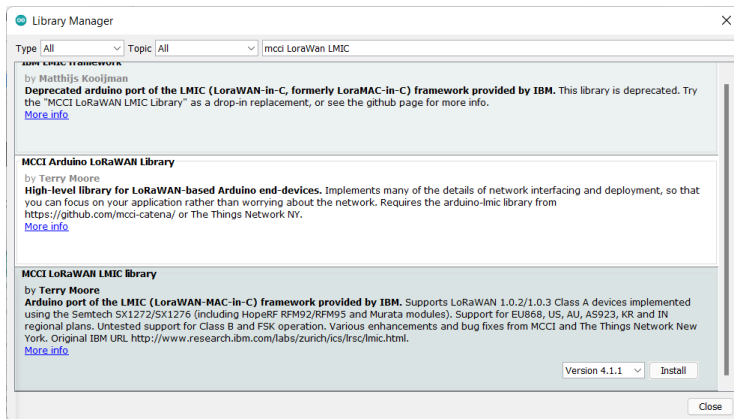


Figure 7.9.15: Arduino – Library Manager

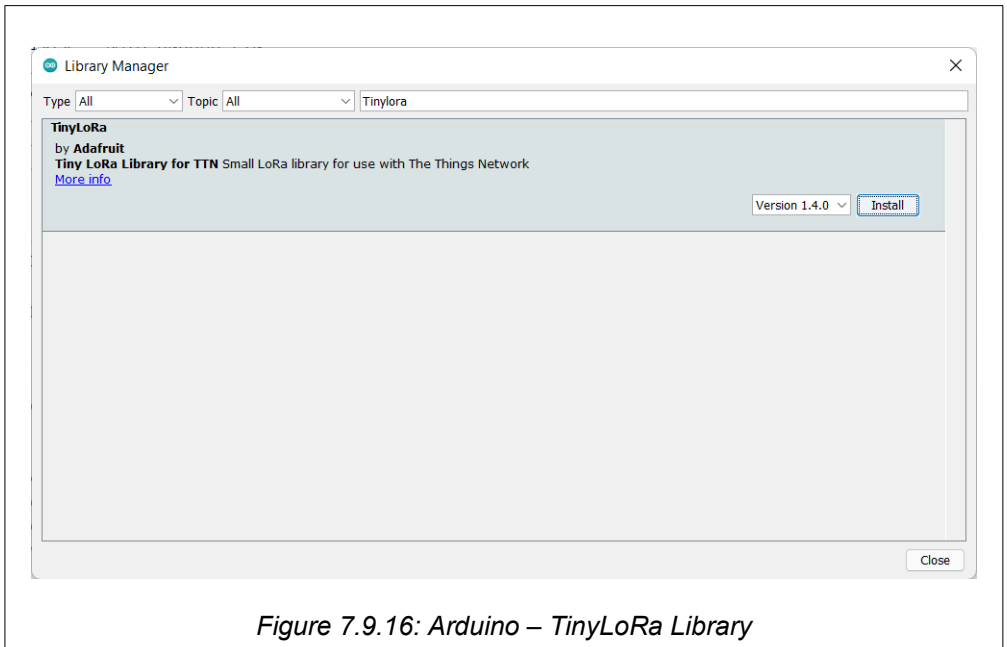
Now you can start with opening the Example => MCCI LoRaWAN LMIC library => ttn-otaa-feather-us915-dht22

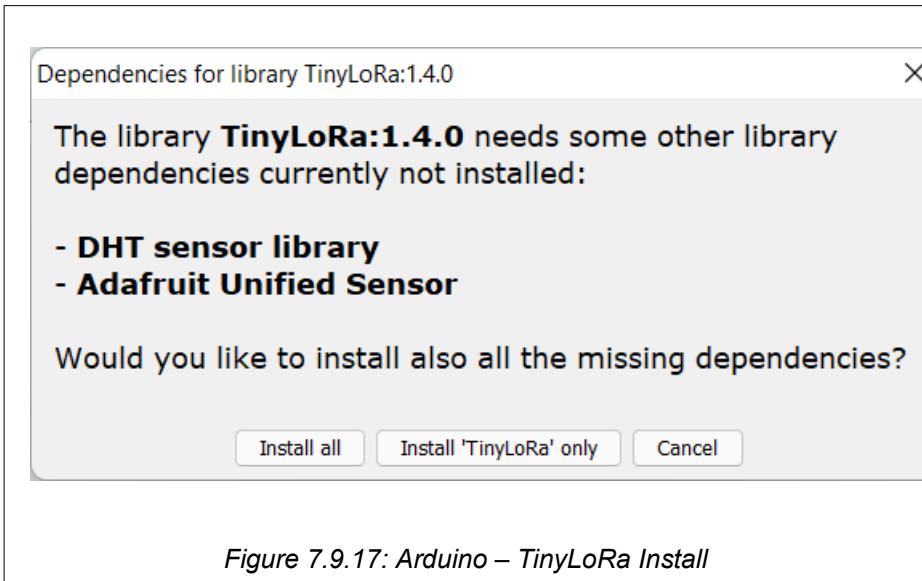
Since we have to modify this file you shall also open the file => open
=> "Arduino\libraries\MCCI_LoRaWAN_LMIC_library\
project_config\ lmic_project_config.h".

Uncomment the "#define CFG_eu868 1" and comment the
"// #define CFG_us915 1" instead.

Verify that your model of Feather 32u4 also uses the SX1276 LoRa based Radio.

For further information, see Ref [60] and [61].





There are some more advanced information about sensor settings in Ref[60].

For troubleshooting it can be good to know the following commands in the command window (ssh):

```
sudo tail -f /var/log/messages
```

```
mosquitto_sub -v -t "#"
```

Bibliography

- [1] Internet of things. [Online; accessed 03-Januari-2022].
URL: https://en.wikipedia.org/wiki/Internet_of_things.
- [2] LoRa. [Online; accessed 03-Januari-2022].
URL: <https://lora-developers.semtech.com/>.
- [3] LoRaWAN. LoRa Alliance [Online; accessed 03-Januari-2022].
URL: <https://lora-alliance.org/about-lorawan/>.
- [4] ChirpStack. [Online; accessed 03-Januari-2022].
URL: <https://www.chirpstack.io>.
- [5] Machine Learning. (Géron, Aurélien, 2016) ISBN:978-1-492-03264-9. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.
- [6] Mesh Networks: Jeferson Rodrigues Cotrim and João Henrique Kleinschmidt, 2020. "LoRaWAN Mesh Networks: A Review and Classification of Multihop Communication". [Online; accessed 03-Januari-2022]. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7435450/pdf/sensors-20-04273.pdf>
- [7] Raspberry Pi. [Online; accessed 03-January-2022].
URL: <https://www.raspberrypi.com/>.
- [8] The Things Network. [Online; accessed 03-January-2022].
URL: <https://www.thethingsnetwork.org/>.
- [9] Arduino. [Online; accessed 03-January-2022].
URL: <https://www.arduino.cc/>.
- [10] Pycom. [Online; accessed 03-January-2022].
URL: <https://pycom.io/>.

[11] Adafruit feather. [Online; accessed 03-January-2022].

URL: <https://www.adafruit.com/>.

[12] Seeed Studio LoRa GPS HAT. [Online; accessed 03-January-2022].

URL: <https://www.seeedstudio.com/Raspberry-Pi-LoRa-GPS-HAT-support-868M-frequency.html>.

[13] Waveshare SX1262 LoRa HAT. [Online; accessed 03-January-2022].

URL: <https://www.waveshare.com/sx1262-868m-lora-hat.htm>.

[14] iC880A-SPI. [Online; accessed 03-January-2022].

URL: <https://shop.imst.de/wireless-modules/lora-products/8/ic880a-spi-lorawan-concentrator-868-mhz>.

[15] Tindie. IMST iC880a LoRaWAN backplane [Online; accessed 03-January-2022].

URL: <https://www.tindie.com/products/iotdevices/imst-ic880a-lorawan-backplane/>.

[16] Waveshare SX1302 LoRaWAN Gateway HAT [Online; accessed 03-January-2022].

URL: <https://www.waveshare.com/product/iot-communication/long-range-wireless/sx1302-lorawan-gateway-hat.htm>.

[17] Get started with MicroPython on Raspberry Pi Pico [Online; accessed 03-January-2022].

URL: <https://www.waveshare.com/product/iot-communication/long-range-wireless/sx1302-lorawan-gateway-hat.htm>.

[18] Halfcare and Everand (2021). Get started with MicroPython on Raspberry Pi Pico. ISBN 9781912047864

[19] Arduino starter kit. [Online; accessed 03-January-2022].

URL: <https://www.arduino.cc/en/Main/ArduinoStarterKit/>.

[20] Chin. R. 2018. Arduino and Raspberry Pi Sensor Projects for the Evil Genius. ISBN 978126001893

[21] 37 Sensor kit tutorial. [Online; accessed 03-January-2022].

URL:

<https://images-eu.ssl-images-amazon.com/images/I/C11rpIfADaS.pdf>

[22] Manual Robotics car kit. [Online; accessed 03-January-2022].

URL: [https://manuals.plus/m/](https://manuals.plus/m/c8a6f9532b6d67dc3c4604ee265f8d5417d6a2bb773d30278634ff45d8669441.pdf)

[c8a6f9532b6d67dc3c4604ee265f8d5417d6a2bb773d30278634ff45d8669441.pdf](https://manuals.plus/m/c8a6f9532b6d67dc3c4604ee265f8d5417d6a2bb773d30278634ff45d8669441.pdf)

[23] Youtube. Everything you need to know about LoRaWAN in 60 minutes [Online; accessed 03-January-2022].

URL: https://www.youtube.com/watch?v=ZsVhYiX4_6o&t=2385s

[24] Wikipedia LoRa [Online; accessed 04-January-2022].

URL: <https://en.wikipedia.org/wiki/LoRa>

[25] AN1200.22 LoRa™ Modulation Basics. Rev 2, May 2015. Semtech Corporation. [Online; accessed 04-January 2022]. URL:

<https://www.semtech.com>

[26] RF wireless World. [Online; accessed 05-January 2022]. URL:

<https://www.rfwireless-world.com/>

[27] RP002-1.0.3 LoRaWAN® Regional Parameters. [Online; accessed 05-January 2022]. URL:

<https://lora-alliance.org/wp-content/uploads/2021/05/RP-2-1.0.3.pdf>

[28] LoRa-Alliance. NetID FAQ [Online; accessed 05-January 2022].

URL:

https://lora-alliance.org/wp-content/uploads/2021/01/20200714_NetID_FAQ.pdf

[29] Semtech. What are LoRa and LoRaWAN [Online; accessed 05-January 2022].

URL:

<https://lora-developers.semtech.com/documentation/tech-papers-and-guides/lora-and-lorawan/>

[30] Stonybrooks. Wireless and Mobile Networks IoT Sensor Networks course [Online; accessed 05-January 2022].

URL: <https://www3.cs.stonybrook.edu/~mdasari/courses/cse570/lora.pdf>

[31] TR007 Developing LoRaWAN devices [Online; accessed 05-January 2022].

URL: <https://lora-alliance.org>

[32] LoRaTools. Air time calculator. [Online; accessed 09-January 2022].

URL: <https://loratools.nl/#/airtime>

[33] LoRa-Alliance. Resource hub. [Online; accessed 09-January 2022].

URL: https://lora-alliance.org/search-site/?_sf_s=LoRaWAN%C2%AE%20Specification%20v

[34] Pycom. [Online; accessed 23-January-2022].

URL: <https://docs.pycom.io/updatefirmware/device/>

[35] Mobilefish.com - Build LoRa gateway using Raspberry Pi 3 model B and Dragino LoRa shield v1.1. [Online; accessed 23-January-2022].

URL: https://www.mobilefish.com/developer/lorawan/lorawan_quickguide_build_lora_gateway.html

[36] Getting Started with Pygate - July 2020 - YouTube [Online; accessed 23-January-2022].

URL: <https://www.youtube.com/watch?v=fYDMz1lmmF0>

[37] Mobilefish.com - LoRa/LoRaWAN tutorial. [Online; accessed 23-January-2022].

URL: https://www.mobilefish.com/developer/lorawan/lorawan_quickguide_tutorial.html

[38] Mobilefish.com - Install and configure ChirpStack components with Semtech UDP packet-forwarder backend. [Online; accessed 23-January-2022].

URL: https://www.mobilefish.com/developer/lorawan/lorawan_quickguide_chirpstack_udp_installation.html

Study of LoRaWAN device and gateway setups

[39] LoRa/LoRaWAN Gateway Kit - Seeed Wiki (seeedstudio.com) [Online; accessed 23-January-2022].

URL:

https://wiki.seeedstudio.com/LoRa_LoRaWan_Gateway_Kit/

[40] LORIoT - The LoRaWAN® Network Server Provider (seeedstudio.com) [Online; accessed 23-January-2022].

URL: <https://www.loriot.io/>

[41] Limitation of Single Channel Gateway - Wiki for Dragino Project [Online; accessed 23-January-2022].

URL: https://wiki.dragino.com/index.php?title=Limitation_of_Single_Channel_Gateway

[42] Core Electronics | Core Electronics Australia (core-electronics.com.au) [Online; accessed 23-January-2022].

URL: <https://core-electronics.com.au/tutorials/pycom/>

[43] Atom (pycom.io) [Online; accessed 23-January-2022].

URL: <https://docs.pycom.io/gettingstarted/software/atom/>

[44] Visual Studio Code (pycom.io) [Online; accessed 23-January-2022].

URL:

<https://docs.pycom.io/gettingstarted/software/vscode/>

[45] Visual Studio Code - Code Editing. Redefined [Online; accessed 23-January-2022].

URL: <https://code.visualstudio.com>

[46] Node.js (nodejs.org) [Online; accessed 23-January-2022].

URL: <https://nodejs.org/en/>

[47] Presenting the temperature, humidity and moisture of my greenhouse in a cloud service - HackMD [Online; accessed 23-January-2022].

URL: <https://hackmd.io/@sMJAvvKqQ6OQ7Y1uLK8WYO/S1dhsfIp>

[48] MicroPython tutorial for the pyboard — MicroPython 1.18 documentation [Online; accessed 23-January-2022].

URL:

<https://docs.micropython.org/en/latest/pyboard/tutorial/index.html>

[49] GitHub - adafruit/Adafruit-GFX-Library [Online; accessed 02-Februari-2022].

URL: <https://github.com/adafruit/Adafruit-GFX-Library>

[50] Adafruit Learning System [Online; accessed 03-Februari-2022].

URL: <https://learn.adafruit.com/>

[51] Overview | Using LoraWAN and The Things Network with Feather | Adafruit Learning System [Online; accessed 03-February-2022].

URL: <https://learn.adafruit.com/the-things-network-for-feather/overview>

[52] KY-013 Analog Temperature Sensor Module - ArduinoModulesInfo [Online; accessed 13-February-2022].

URL: <https://arduinomodules.info/ky-013-analog-temperature-sensor-module/>

[53] https://en.wikipedia.org/wiki/Steinhart-Hart_equation [Online accessed 13-February-2022].

URL: https://en.wikipedia.org/wiki/Steinhart-Hart_equation

[54] Home | The Things Stack for LoRaWAN (thethingsindustries.com) [Online accessed 13-February-2022].

URL: <https://www.thethingsindustries.com/docs/>

[55] ChirpStack. Gateway-bridge configuration. [Online accessed 20-February-2022].

URL: <https://www.chirpstack.io/gateway-bridge/install/config/>

[56] Introduction - ChirpStack open-source LoRaWAN Network Server. [Online accessed 6-Mars-2022].

URL: <https://www.chirpstack.io/gateway-os/install/raspberrypi/>

[57] balenaEtcher - Flash OS images to SD cards & USB drives [Online accessed 6-Mars-2022].

URL: <https://www.balena.io/etcher/>

[58] iC880A-SPI Documents - Wireless Solutions (wireless-solutions.de) [Online accessed 6-june-2022].

URL: <https://wireless-solutions.de/downloadfile/ic880a-spi-documents/>

[59] Connecting a gateway - ChirpStack open-source LoRaWAN[®] Network Server [Online accessed 6-June-2022].

URL: <https://www.chirpstack.io/project/guides/connect-gateway/>

[60] Devices - ChirpStack open-source LoRaWAN[®] Network Server [Online accessed 6-June-2022].

URL: <https://www.chirpstack.io/application-server/use/devices/>

[61] chirpstack-application-server/LICENSE at master · brocaar/chirpstack-application-server · GitHub [Online accessed 6-June-2022].

URL: <https://github.com/brocaar/chirpstack-application-server/blob/master/LICENSE>

[62] Arduino Code | Using LoraWAN and The Things Network with Feather | Adafruit Learning System [Online accessed 6-June-2022].

URL: <https://learn.adafruit.com/the-things-network-for-feather/arduino-code>

[63] Device Classes | The Things Network [Online accessed 26-June-2022].

URL:
<https://www.thethingsnetwork.org/docs/lorawan/classes/>